

1. INTRODUCTION

Artificial intelligence and machine learning are driving some of the biggest changes in tech right now. They're everywhere—from the way your phone recognizes faces in photos to how voice assistants understand what you're saying. But if you strip away all the complexity, you get to this basic building block inside every neural network: the perceptron. Frank Rosenblatt came up with it back in 1958. It's pretty simple at its core—it takes several weighted inputs, computes their weighted sum, then pushes out an output, just like a biological neuron.

Now, with everyone pushing for faster and more energy-efficient AI hardware, it's not enough to just simulate these networks in software. Engineers are moving toward building neural network pieces straight onto silicon, right at the transistor level. This isn't just a fun challenge—it actually makes a big difference in speed, power use, and how much space the circuit takes up on the chip. Those three things are a big deal in VLSI design.

This project lays out a complete transistor-level design for a high-speed perceptron circuit, and everything's been built and tested in Cadence Virtuoso. Here's how it works: the perceptron takes two 4-bit signed numbers (in two's complement form), multiplies each by a programmable weight using arithmetic right-shifts, and then adds them together through a signed binary adder. Out the other end comes a 5-bit result.

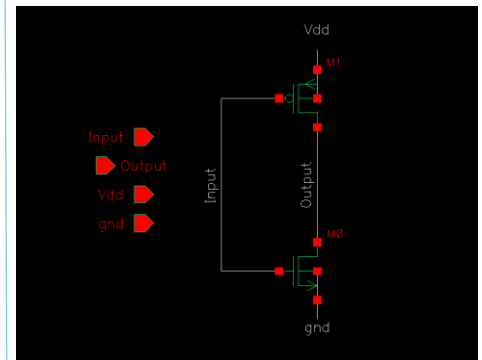
The design is layered—starting from basics, like CMOS logic gates (inverters, NANDs, NORs, ANDs, ORs, and XORs), and building up from there. Those basic gates come together to create blocks like a 2:1 multiplexer, a 4-bit barrel shifter that acts as the multiplier, and a signed 4-bit adder. The complete system operates at a 1 GHz clock, so the whole system is fast enough to keep up with the demands of real neural network hardware accelerators.

2. BUILDING BLOCKS (Logic Gates)

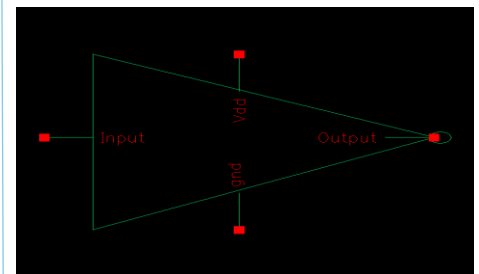
2.1 CMOS Inverter (NOT Gate)

The CMOS inverter is the most fundamental building block in digital circuit design. It consists of a single PMOS transistor connected to VDD and a single NMOS transistor connected to GND, with both gates tied to the input and both drains tied to the output. When the input is logic HIGH, the NMOS turns ON and PMOS turns OFF, pulling the output to GND. When the input is logic LOW, the PMOS turns ON and NMOS turns OFF, pulling the output to VDD. The CMOS inverter exhibits rail-to-rail output swing, zero static power dissipation, and high noise margins, making it the ideal buffer and logic complement element. ($Y = \bar{A}$)

NOT GATE



schematic

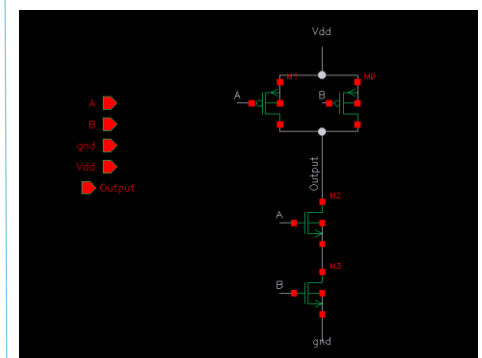


symbol

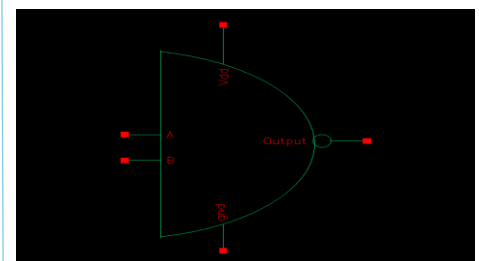
2.2 NAND Gate

A CMOS NAND gate uses two NMOS transistors connected in series at the bottom (pull-down network) and two PMOS transistors connected in parallel at the top (pull-up network). It only outputs LOW when both inputs are HIGH—at that point, both NMOS transistors turn on, and both PMOS transistors switch off. Any other input, at least one PMOS conducts, which drives the output HIGH. NAND gate is called the universal gate in CMOS tech because it's so efficient at the transistor level and you can build any Boolean function from it. $Y = \overline{A \cdot B}$

NAND GATE



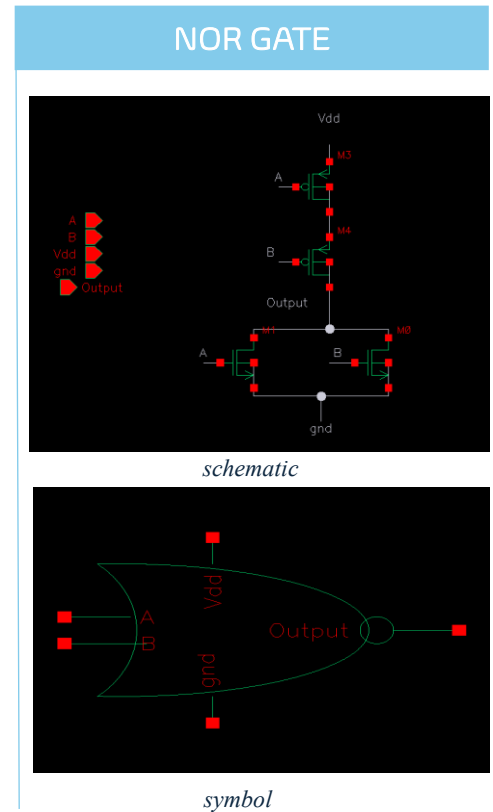
schematic



symbol

2.3 NOR Gate

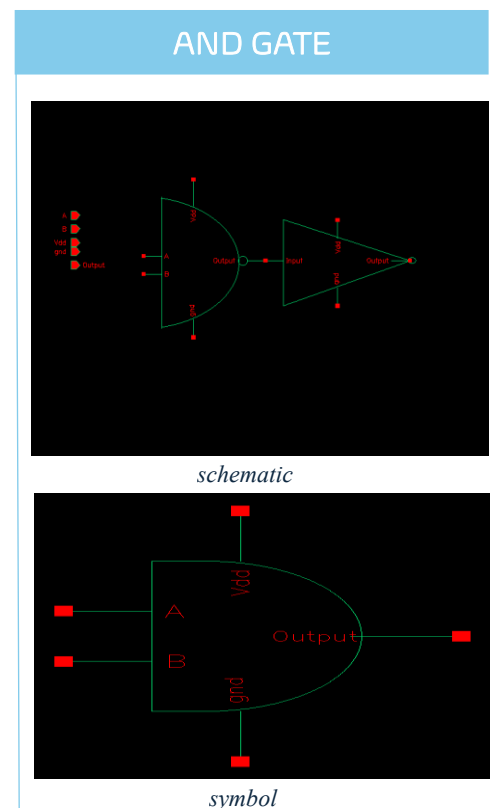
The CMOS NOR gate consists of two parallel NMOS transistors in the pull-down network and two series PMOS transistors in the pull-up network. The output is HIGH only when both inputs are LOW, allowing both series PMOS transistors to conduct. If either input goes HIGH, the corresponding NMOS transistor conducts and the output is pulled LOW. Due to the series PMOS configuration, NOR gates suffer from higher resistance in the pull-up path compared to NAND gates, making NAND gates generally preferred in CMOS design. Nevertheless, NOR gates serve as the second universal gate and are essential in many logic implementations. $Y = (\overline{A + B})$. To optimize performance, the bulk terminals are biased to VDD for PMOS and GND for NMOS. This eliminates the Body Effect by maintaining $V_{SB} = 0$, ensuring a stable threshold voltage (V_{th}). Consequently, this prevents propagation delay degradation, which is vital for maintaining high-speed switching at the target 1 GHz frequency.



2.4 AND Gate

The AND gate can be realizable in CMOS by cascading a NAND gate with an inverter. Since a pure CMOS AND gate cannot be implemented directly without inversion, the NAND-INV combination is the standard approach.

The output is HIGH only when all inputs are simultaneously HIGH. In our project, the AND gate is used as a logic building block within the adder and control logic of the perceptron. $Y = (A \cdot B)$

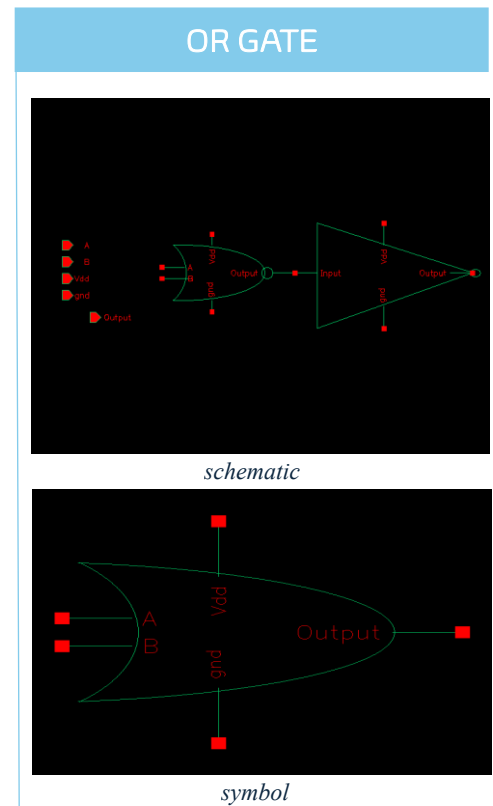


2.5 OR Gate

Similarly to the AND gate, the OR gate in CMOS is implemented by cascading a NOR gate followed by an inverter.

The output is HIGH when at least one input is HIGH.

The OR gate plays a critical role in the sum and carry logic of the full adder used in the perceptron's accumulation stage. $Y = (A + B)$

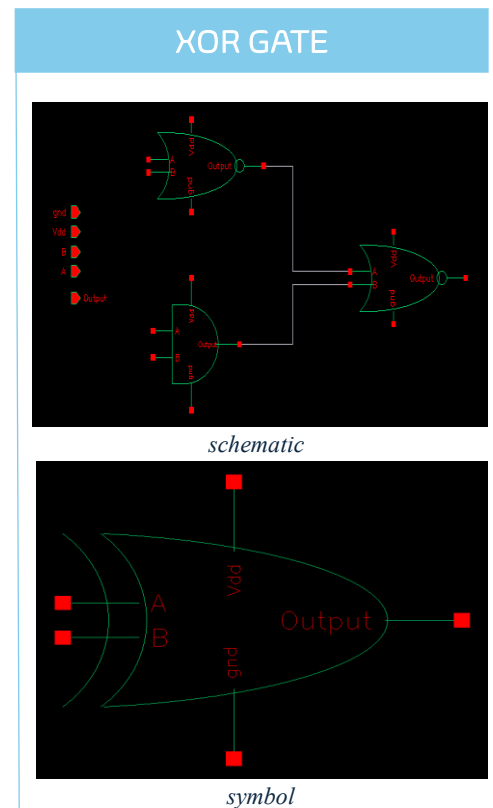


2.6 XOR Gate

The XOR (Exclusive-OR) gate produces a HIGH output when its inputs are different and a LOW output when they are identical.

It is implemented using a combination of NAND and NOR gates at the transistor level, as visible in the schematic.

The XOR gate is the most critical building block in this project as it forms the core of the **Full Adder sum logic**, where the sum bit is computed as $A \oplus B \oplus \text{Cin}$.



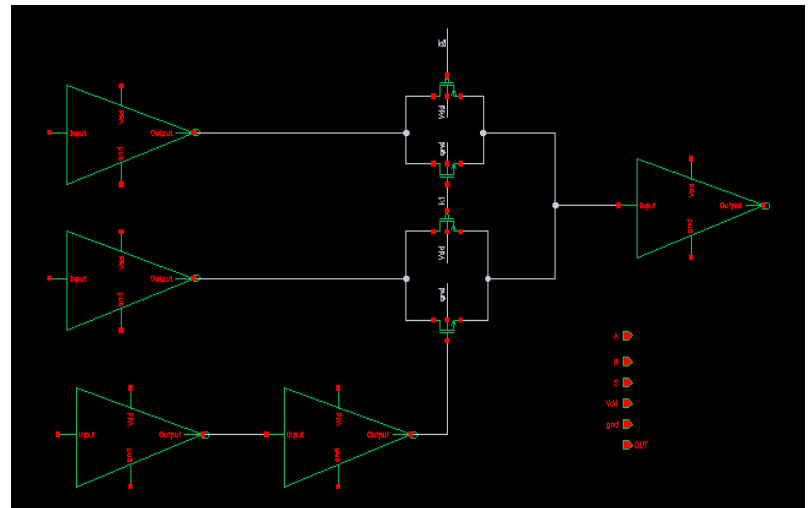
3. BUILDING BLOCKS (Circuit Design)

3.1 2:1 Multiplexer (MUX)

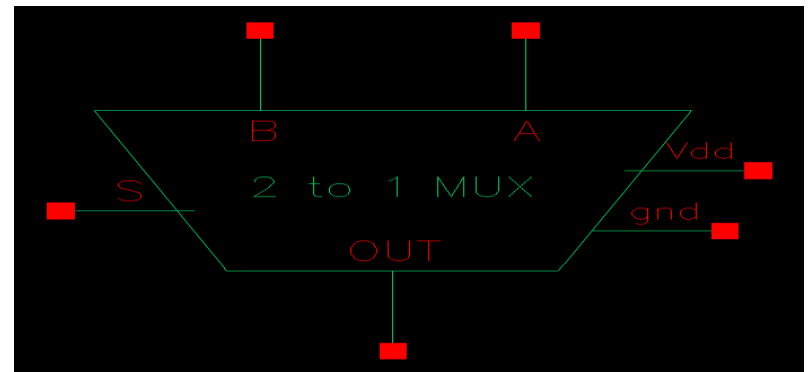
The 2:1 Multiplexer forms the fundamental switching element of the barrel shifter. The implemented MUX is based on a **transmission gate topology**, which is the most efficient CMOS realization in terms of transistor count and propagation delay. As shown in the schematic, the circuit consists of two transmission gates each formed by a complementary NMOS/PMOS pair connected in parallel controlled by the selection signal S and its complement. Input buffers are placed at both data inputs to restore signal levels and drive the transmission gates properly, while an output inverter serves as a buffer to strengthen the output drive capability and isolate the output node from the high-impedance transmission gate.

When $S = 0$, the upper transmission gate conducts, passing input I_0 to the output. When $S = 1$, the lower transmission gate conducts, selecting input I_1 . The transmission gate MUX offers near-ideal switching behavior with minimal voltage drop across the pass transistors, making it well-suited for high-speed operation at 1 GHz.

2:1 MUX



schematic



symbol

The 2:1 MUX is a digital selector switch. It has:

- 2 data inputs: A and B
- 1 selection input: S
- 1 output: $Y = \bar{S} \cdot A + S \cdot B$

Its only job is: "based on S, which input do I pass to the output?"

3.1.1 [The two data inputs arrive (A and B)]

The moment A and B enter the circuit, they don't go directly to the transmission gates. They first hit input inverters (buffers). Why?

*Because the signal coming from outside might be weak — low voltage swing, not enough current drive.
The inverter:*

- *Restores the signal to full VDD or GND level*
- *Strengthens the current drive capability*
- *Isolates the previous stage from loading effects*

So after this step, we have clean, strong versions of A and B ready to be switched.

3.1.2 [The selection signal S enters and gets inverted]

At the same time, the selection signal S enters a dedicated inverter to produce $\bar{S}$ (S-bar, the complement of S).

Now the circuit has both S and $\bar{S}$ available. This is critical because the transmission gate needs both the signal and its complement to control the NMOS and PMOS transistors simultaneously.

3.1.3 [The Transmission Gates – The Heart of the MUX]

Each transmission gate consists of one NMOS and one PMOS transistor connected in parallel between the input and the output node

Why both NMOS and PMOS in parallel?

- NMOS passes logic 0 perfectly but degrades logic 1 (threshold voltage drop)
- PMOS passes logic 1 perfectly but degrades logic 0
- Together they cover the full voltage range from 0 to V_{DD} with no degradation

This is why the transmission gate is far superior to using just a single NMOS or PMOS switch.

3.1.4 [What happens when $S = 0$?]

- TG_1 (for A): NMOS gate = $\overline{S}$, PMOS gate = S
- TG_2 (for B): NMOS gate = S , PMOS gate = $\overline{S}$

When $S = 0$ ($\overline{S} = 1$):

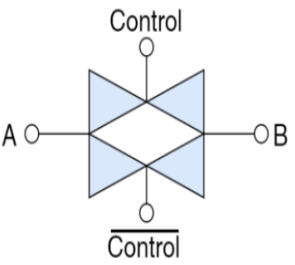
TG_1 (for A):

- NMOS gate = $\overline{S} = 1 \rightarrow \text{ON}$
- PMOS gate = $S = 0 \rightarrow \text{ON}$
- TG_1 **CLOSED** $\rightarrow$ A flows to output node

TG_2 (for B):

- NMOS gate = $S = 0 \rightarrow \text{OFF}$
- PMOS gate = $\overline{S} = 1 \rightarrow \text{OFF}$
- TG_2 **OPEN** $\rightarrow$ B is blocked

Result: $Y = A$

Symbol	Truth Table		
	Control	A	B
	1	0	0
	1	1	1
	0	0	Hi-Z
	0	1	Hi-Z

3.1.5 [What happens when $S = 1$?

TG₁ (for A):

- NMOS gate = $\overline{S} = 0 \rightarrow \text{OFF}$
- PMOS gate = $S = 1 \rightarrow \text{OFF}$
- **TG₁ OPEN** $\rightarrow$ A is blocked

TG₂ (for B):

- NMOS gate = $S = 1 \rightarrow \text{ON}$
- PMOS gate = $\overline{S} = 0 \rightarrow \text{ON}$
- **TG₂ CLOSED** $\rightarrow$ B flows to output node

Result: $Y = B$

3.1.6 [The output node]

After one of the two transmission gates passes its signal, the selected signal arrives at the output node. But at this point the signal might still be slightly weak due to the resistance of the transmission gate. So a final output inverter is placed at the output to:

- *Restore full signal swing*
- *Provide strong drive current for the next stage*
- *Prevent the output from being loaded by whatever comes after the MUX*

Since this inverter inverts the signal, the input signals were pre-inverted at the input stage to compensate, so the final output Y is the correct non-inverted version of the selected input.

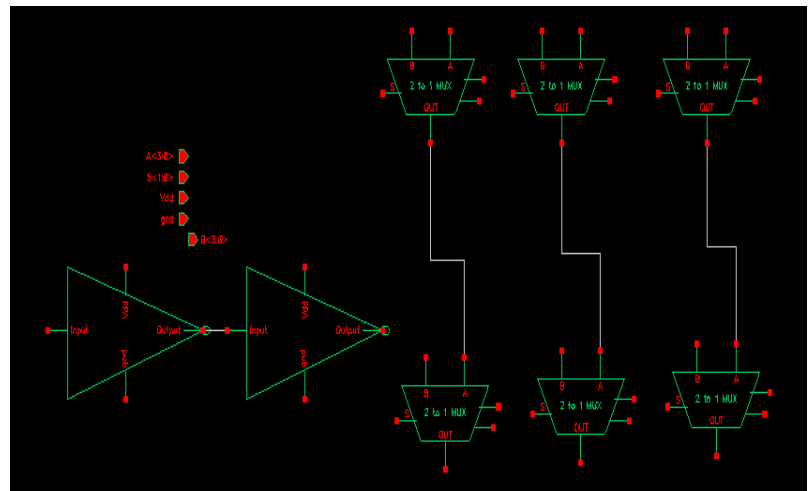
3.2 4-bit Barrel Shifter (Multiplier)

The 4-bit barrel shifter does the weighted multiplication the perceptron needs. It takes a 4-bit signed input and shifts it right by 0, 1, 2, or 3 bits, depending on the 2-bit weight input $W = \{S_1, S_0\}$. This ends up multiplying the input by 1, 0.5, 0.25, or 0.125.

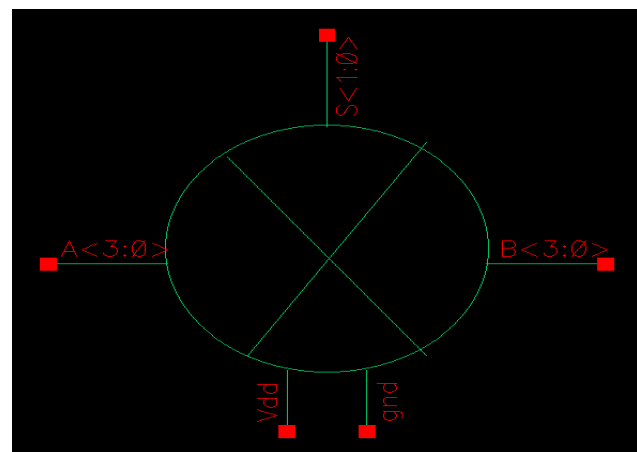
Here's how it's built: it uses two stages of 2-to-1 multiplexers. The first stage, controlled by S_0 , either leaves the number alone or performs an arithmetic right shift by one bit. The second stage, using S_1 , either leaves it as is or shifts everything two more bits to the right. With just six MUXes wired up in this cascaded way, all four possible shift values are easy to get, and the design stays compact and tidy.

One thing you really have to watch out for is the sign extension. Since inputs are two's complement signed numbers, any arithmetic right shift needs to fill the new leftmost (MSB) spots with the original sign bit (A_3), not with zeros. That's the only way to preserve the numeric polarity in 2's complement when you shift them. So the design just connects A_3 straight to the output's MSB (B_3), making sure the sign bit always gets copied, no matter how much you shift.

4-bit Barrel Shifter



schematic



symbol

What is the Barrel Shifter?

The Barrel Shifter in this project acts as the multiplier of the perceptron. But instead of performing actual binary multiplication (which requires complex hardware), it performs arithmetic right shifts — a much simpler and faster operation that achieves the same result for powers of 2.

It has:

- 4-bit signed input: $A = \{A_3, A_2, A_1, A_0\}$ where A_3 is the MSB (sign bit)
- 2-bit weight/control input: $W = \{S_1, S_0\}$
- 4-bit signed output: $B = \{B_3, B_2, B_1, B_0\}$

Its job is simple: shift A to the right by 0, 1, 2, or 3 positions based on W.

Why Right Shift = Multiplication?

In binary, shifting a number right by n bits is mathematically equivalent to dividing by 2^n , which is the same as multiplying by $1/2^n$

So instead of building a complex multiplier, we just shift bits around which is much faster, much simpler.

W (S_1S_0)	Shift Amount	Multiplication Factor
00	0 bits	$\times 1$
01	1 bit right	$\times 0.5$
10	2 bits right	$\times 0.25$
11	3 bits right	$\times 0.125$

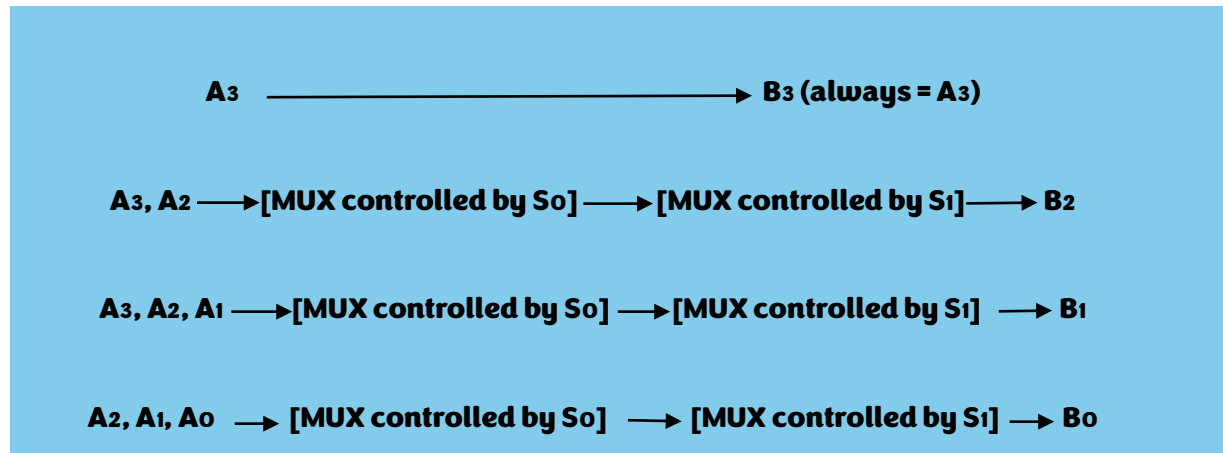
Architecture — Two Stages of MUXes

The barrel shifter is built from two cascaded stages of 2:1 MUXes:

Stage 1 (controlled by S_0): **Stage 2 (controlled by S_1):**
Either shift by 0 or 1 bit **Either shift by 0 or 2 more bits**

This is the key insight: instead of building a separate circuit for each of the 4 possible shifts, we cascade two stages.

The full structure for each output bit



3.2.1 [Input A₃ A₂ A₁ A₀ and weight W = {S₁ S₀} arrives]

The 4-bit input enters the first stage. S₀ (LSB of weight) controls Stage 1, S₁ (MSB of weight) controls Stage 2.

3.2.2 [Stage 1 – S₀ decides "shift by 0 or 1?"]

Each MUX in Stage 1 chooses between two adjacent bits:

Output of Stage 1	If S ₀ = 0 (no shift)	If S ₀ = 1 (shift by 1)
Bit 3	A ₃	A ₃ (sign preserved always)
Bit 2	A ₂	A ₃
Bit 1	A ₁	A ₂
Bit 0	A ₀	A ₁

3.2.3 [Stage 2 – S_1 decides "shift by 0 more or 2 more?]

The output of Stage 1 feeds into Stage 2. Now S_1 chooses between the Stage 1 output and a version shifted by 2 more positions:

Output of Stage 2 (B)	If $S_1 = 0$ (no extra shift)	If $S_1 = 1$ (shift by 2 more)
B₃	A₃	A₃ (sign always)
B₂	Stage1[2]	A₃
B₁	Stage1[1]	A₃
B₀	Stage1[0]	Stage1[2]

3.2.4 [Final output $B_3 B_2 B_1 B_0$ is ready]

The combined effect of both stages gives the correct arithmetic right shift for all 4 weight values.

3.2.5 [Examples]

Example 1:

$W = 00$ (No shift, $\times 1$)

Input: $X = 0110 = +6$

$S_0 = 0, S_1 = 0 \rightarrow$ No shift in either stage

Stage 1 ($S_0=0$): passes A as-is $\rightarrow 0110$

Stage 2 ($S_1=0$): passes Stage1 as-is $\rightarrow 0110$

Output B = 0110 = +6

Result: $6 \times 1 = 6$

Example 2:

$W = 01$ (1-bit right shift, $\times 0.5$)

Input: $X = 0110 = +6$

$S_0 = 1, S_1 = 0$

Stage 1 ($S_0=1$): shift right by 1

$B_3 = A_3 = 0$ (sign preserved)

$B_2 = A_3 = 0$, $B_1 = A_2 = 1$, $B_0 = A_1 = 1$

Stage 1 output = 0011

Stage 2 ($S_1=0$): no extra shift $\rightarrow 0011$

Output B = 0011 = +3

Result: $6 \times 0.5 = 3$

Example 3:

$W = 10$ (2-bit right shift, $\times 0.25$)

Input: $X = 0110 = +6$

$S_0 = 0, S_1 = 1$

Stage 1 ($S_0=0$): no shift $\rightarrow 0110$

Stage 2 ($S_1=1$): shift right by 2

$B_3 = A_3 = 0$ (sign preserved)

$B_2 = A_3 = 0$

$B_1 = \text{Stage1}[3] = 0$, $B_0 = \text{Stage1}[2] = 1$

Output = 0001

Result: $6 \times 0.25 = 1.5 \rightarrow \text{floor} = 1$ (floor division due to right shift)

Example 4:

$W = 11$ (3-bit right shift, $\times 0.125$)

Input: $X = 0110 = +6$

$S_0 = 1, S_1 = 1$

Stage 1 ($S_0=1$): shift right by 1

$B_3 = 0, B_2 = 0, B_1 = 1, B_0 = 1$

Stage 1 output = 0011

Stage 2 ($S_1=1$): shift right by 2 more

$B_3 = 0$ (sign) , $B_2 = 0$ (sign fill)

$B_1 = 0$ (sign fill) , $B_0 = \text{Stage1}[2] = 0$

Output = 0000

Result: $6 \times 0.125 = 0.75 \rightarrow \text{floor} = 0$

Example 5:

Negative number, $W = 01$ ($\times 0.5$)

Input: $X = 1110 = -2$ (two's complement)

$S_0 = 1, S_1 = 0$

Stage 1 ($S_0=1$): shift right by 1, fill with sign bit ($A_3=1$)

$B_3 = 1$ (sign preserved)

$B_2 = A_3 = 1$ (fill with sign)

$B_1 = A_2 = 1$, $B_0 = A_1 = 1$

Stage 1 output = 1111

Stage 2 ($S_1=0$): no extra shift $\rightarrow 1111$

Example 6:

Negative number, $W = 10$ ($\times 0.25$)

Input: $X = 1100 = -4$ (two's complement)

$S_0 = 0, S_1 = 1$

Stage 1 ($S_0=0$): no shift $\rightarrow 1100$

Stage 2 ($S_1=1$): shift right by 2, fill with sign bit ($A_3=1$)

$B_3 = 1$ (sign) , $B_2 = 1$ (sign fill)

$B_1 = 1$ (sign fill) , $B_0 = \text{Stage1}[2] = 1$

Output = $1111 = -1$

Result: $-4 \times 0.25 = -1$

Example 7:

Negative number, $W = 11$ ($\times 0.125$)

Input: $X = 1000 = -8$ (two's complement)

$S_0 = 1, S_1 = 1$

Stage 1 ($S_0=1$): shift right by 1

$B_3 = 1, B_2 = 1, B_1 = 1, B_0 = 0$

Stage 1 output = 1110

Stage 2 ($S_1=1$): shift right by 2

$B_3 = 1$ (sign)

$B_2 = 1$ (sign fill)

$B_1 = 1$ (sign fill)

$B_0 = \text{Stage1}[2] = 1$

Output = $1111 = -1$

Result: $-8 \times 0.125 = -1$

3.3 Full Adder / Carry Lookahead Adder (CLA)

The full adder sits at the heart of the adder block. You feed it three single-bit inputs A, B, and the carry-in (C_{in}) and it gives you back two outputs: the Sum and the carry-out (C_{out}). It follows these Boolean equations:

$$\text{Sum} = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + C_{in}(A \oplus B)$$

To make it work at the transistor level, everything is built with basic CMOS gates, just the standard XOR, AND, and OR gates. The Sum output comes from wiring up two XOR gates in a row. For C_{out} , you mix AND and OR gates together. Laying out the circuit like this keeps things consistent and easier to reuse if you need to build bigger adders later on.

Instead of sticking with a typical Ripple Carry Adder (RCA), this design uses a Carry Lookahead Adder (CLA) for much faster performance. With an RCA, each bit has to wait for the carry from the bit before it, which slows things down a lot (worst case delay) especially if you're aiming for 1 GHz speeds. The CLA sidesteps this whole issue. It figures out all the carry signals at once with special lookahead logic, so you don't get that long chain of delays.

The CLA is built around two intermediate signals defined for each bit position i :

- **Generate:** $G_i = A_i \cdot B_i$ (carry is generated regardless of C_{in})
- **Propagate:** $P_i = A_i \oplus B_i$ (carry is propagated from C_{in})

The carry at each stage is then expressed as:

- $C_1 = G_0 + P_0 \cdot C_0$
- $C_2 = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0$
- $C_3 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0$
- $C_4 = G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0$

The design works in layers. The CLA_bit cell figures out the Generate and Propagate signals for each bit. At the next level, the CLA_logic cell handles all the lookahead carry equations. Put them together, then we get the full CLA block a 4-bit adder that calculates the sum and the final carries. To support signed 2's complement arithmetic, an additional XOR gate is implemented between (P_3 and C_4). This logic serves as an overflow detection mechanism, ensuring the perceptron correctly handles signed values. By utilizing this architecture, the design achieves the 5-bit signed output required for the perceptron's next stage while maintaining a critical path delay of $O(\log n)$, making it well-suited for the project's 1 GHz target frequency.

3.3.1 [1-Bit Carry Lookahead Cell (CLA Bit)]

The CLA_Bit block serves as the fundamental 1-bit processing unit for the Carry Lookahead Adder (CLA). Unlike a standard Full Adder, which calculates the carry-out internally and causes a ripple effect, the CLA_Bit isolates the sum calculation from carry generation. It calculates localized Generate (*G*) and Propagate (*P*) signals, which are then passed to the external Carry Lookahead logic block to compute carries in parallel. Finally, it uses the incoming carry to compute the final Sum.

Pin Interface

Inputs:

- **A:** 1-bit operand input.
- **B:** 1-bit operand input.
- **C:** 1-bit Carry-in signal from the carry lookahead network.
- **Vdd / gnd:** Power supply and ground pins.

Outputs:

- **G (Generate):** Indicates if a carry is generated at this specific bit position.
- **P (Propagate):** Indicates if an incoming carry will be propagated across this bit position.
- **Sum:** The final 1-bit addition result for this position.

Logic Design and Architecture

The schematic utilizes custom transistor-level CMOS gates to implement the standard carry lookahead Boolean logic:

- **Generate Logic:** The Generate signal is produced using a 2-input **AND** gate connected to inputs A and B. A carry is generated only when both input bits are logic HIGH.

$$G = A \cdot B$$

- **Propagate Logic:**

The Propagate signal is produced using a 2-input **XOR** gate connected to inputs A and B. A carry is propagated through this bit if one (and only one) of the input bits is logic HIGH.

$$P = A \oplus B$$

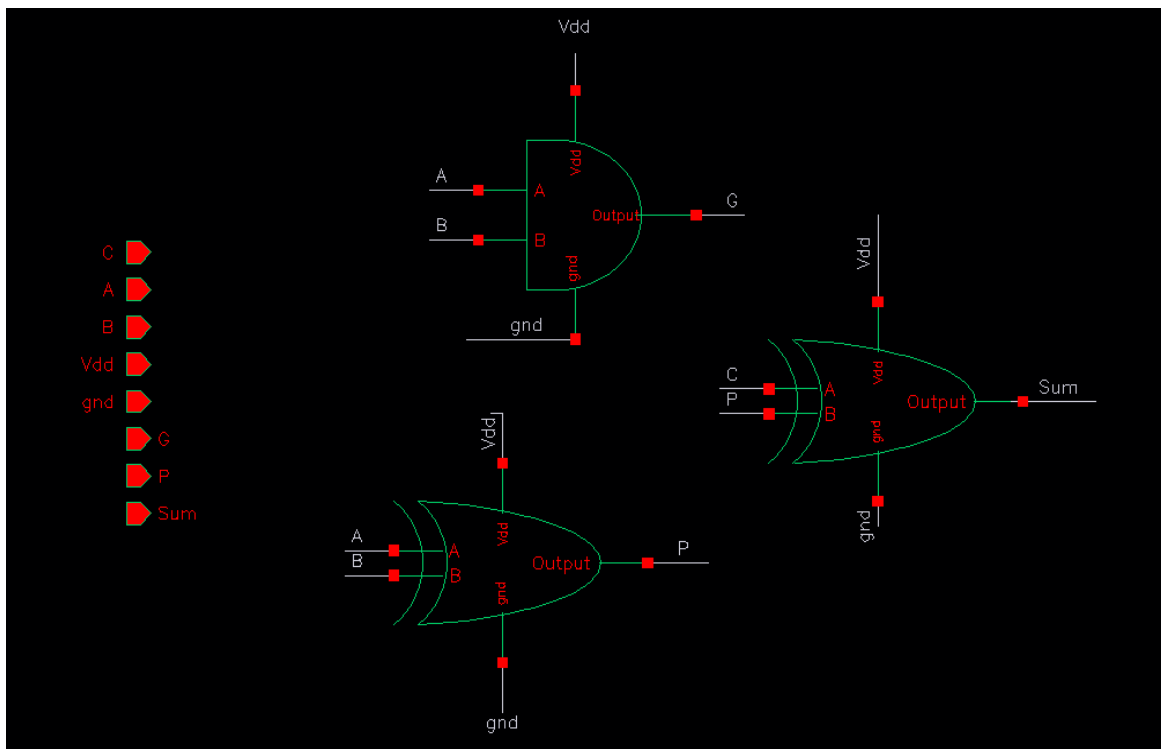
- **Sum Logic:**

The final sum is computed by chaining a second 2-input **XOR** gate. This gate takes the previously computed Propagate signal (**P**) and the incoming Carry bit (**C**) as its inputs.

$$Sum = A \oplus B \oplus C$$

Role of P and G in Subsequent Blocks

The outputs **P** and **G** from each of the four CLA_Bit blocks are exported directly to the centralized CLA_CARRY block. Instead of waiting for a carry to ripple through each bit, the CLA_CARRY block looks at all the **P** and **G** signals simultaneously. It uses them to calculate the exact carry-in (**C1**, **C2**, **C3**, and **C4**) for every bit position in parallel, significantly reducing the overall propagation delay of the adder.



CLA_Bit Block

3.3.2 [CLA_Carry block]

This block pre-calculates the carry signals for all stages simultaneously based on two intermediate signals: Generate (G_i) and Propagate (P_i). By computing the carries in parallel, the total propagation delay is reduced to a constant number of gate levels, regardless of the adder's size. This efficiency is critical for achieving the high-frequency requirements of modern processors and high-speed signal processing units.

3.3.2.1 [[Carry 1 (C1) Generation Logic]]

This section of the CLA_CARRY block computes the first carry-out signal **C1**. Instead of waiting for a ripple-carry operation, it calculates **C1** simultaneously based on the initial carry-in to the system and the localized signals from the 0th bit position.

Interface Signals

Inputs:

- **C0**: The initial Carry-in signal to the 4-bit adder.
- **P[0]**: The Propagate signal generated by the 0th CLA_Bit block.
- **G[0]**: The Generate signal produced by the 0th CLA_Bit block.

Output:

- **C1**: The computed carry signal that will be fed into the 1st CLA_Bit block (and used in subsequent carry calculations).

Logic Design and Architecture

The schematic constructs the fundamental carry lookahead equation for the first bit. The logic dictates that a carry is passed to the next bit if the current bit explicitly *generates* a carry, OR if it *propagates* an existing carry-in.

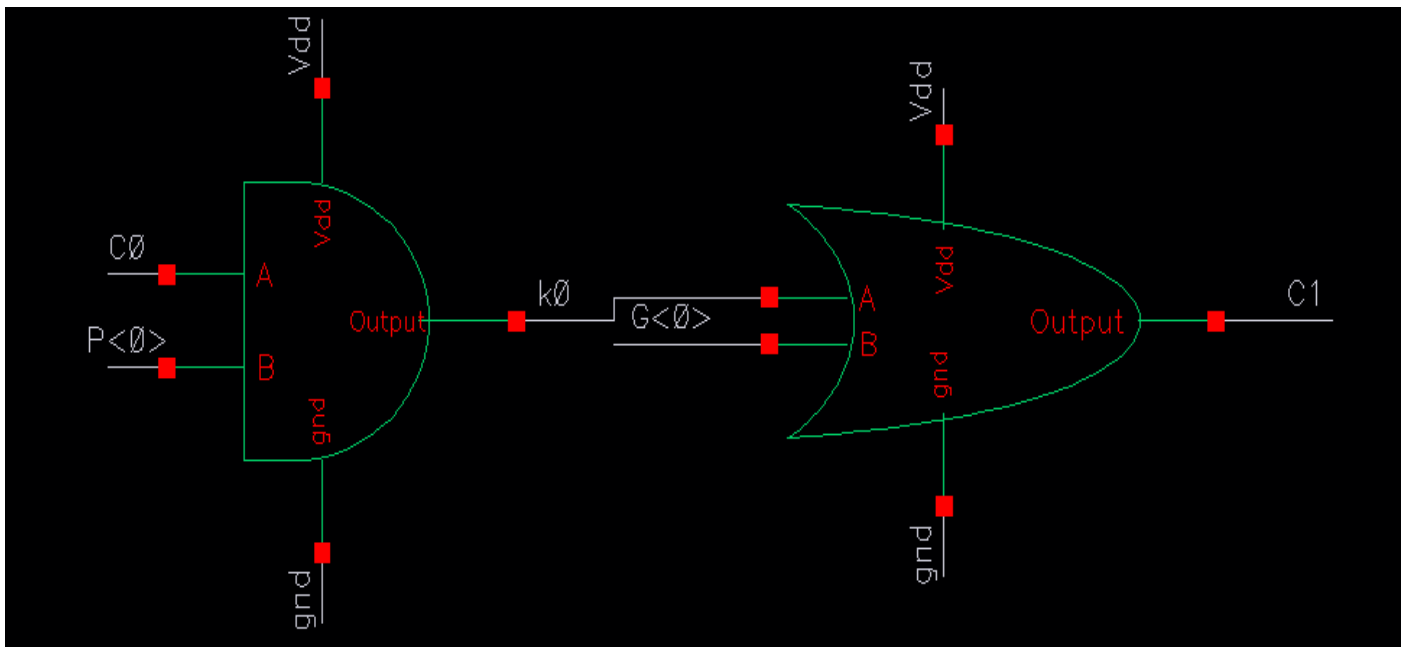
- **AND Logic:** A 2-input AND gate takes $C0$ and $P[0]$ as inputs. This creates an intermediate signal $K0$ representing whether an initial carry exists and is allowed to propagate through the 0th bit.

$$K0 = P[0] \cdot C0$$

- **OR Logic:** A 2-input OR gate takes the result from the AND gate $K0$ and the $G[0]$ signal. This finalizes the logic, confirming that $C1$ is HIGH if a carry was generated locally $G[0]$, or propagated $P[0] \cdot C0$

Boolean Equation

$$C1 = G0 + P0 \cdot C0$$



Carry 1 Generation Block

3.3.2.2 [[Carry 2 (C2) Generation Logic]]

This section computes the second carry-out signal, **C2**. Building upon the carry-lookahead principle, **C2** is calculated directly from the initial carry-in (**C0**) and the generate/propagate signals of the first two bit positions, bypassing the need to wait for **C1** to ripple through a standard adder.

Interface Signals

Inputs:

- **C0**: The initial Carry-in signal.
- **P[0], P[1]**: Propagate signals from the 0th and 1st CLA_Bit blocks.
- **G[0], G[1]**: Generate signals from the 0th and 1st CLA_Bit blocks.

Output:

- **C2**: The computed carry signal fed into the 2nd CLA_Bit block.

Logic Design and Architecture

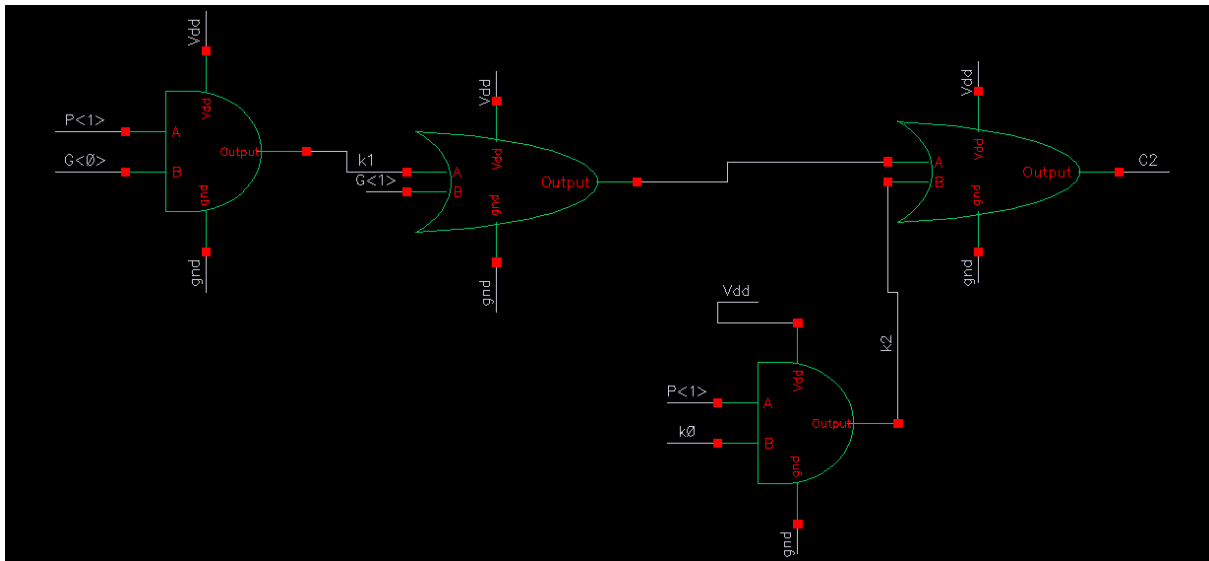
The schematic maps directly to the expanded Boolean equation for the second carry bit. To build the required logic using standard gates, the circuit utilizes a cascaded structure:

- **AND Logic**: A 2-input AND gate computes the term for a carry generated at bit 0 and propagated across bit 1 ($P[1] \cdot G[1]$).
 - To compute the propagation of the initial carry-in, a 2-input AND gate first evaluates $P[0] \cdot C0$. The result is then fed into a second 2-input AND gate along with $P[1]$, effectively calculating the three-variable term $(P[1] \cdot P[0] \cdot C0)$.
- **OR Logic**: A 3-input OR gate collects these conditions. It takes $G[1]$ directly, along with the outputs of the two AND logic branches. If any of these three conditions are met, **C2** is driven HIGH.

Boolean Equation

The standard formula:

$$C2 = G1 + P1 \cdot G0 + P1 \cdot P0 \cdot C0$$



Carry 2 Generation Block

3.3.2.3 [[Carry 3 (C3) Generation Logic]]

This section computes the third carry-out signal, **C3**. Building upon the carry-lookahead principle, **C3** is calculated directly from the initial carry-in (**C0**) and the generate/propagate signals of the first three bit positions, eliminating the ripple-carry delay from **C1** and **C2**.

Interface Signals

Inputs:

- **C0**: The initial Carry-in signal.
- **P[0], P[1], P[2]**: Propagate signals from the 0th, 1st, and 2nd CLA_Bit blocks.
- **G[0], G[1], G[2]**: Generate signals from the 0th, 1st, and 2nd CLA_Bit blocks.

Output:

- **C3:** The computed carry signal fed into the 3rd CLA_Bit block.

Logic Design and Architecture

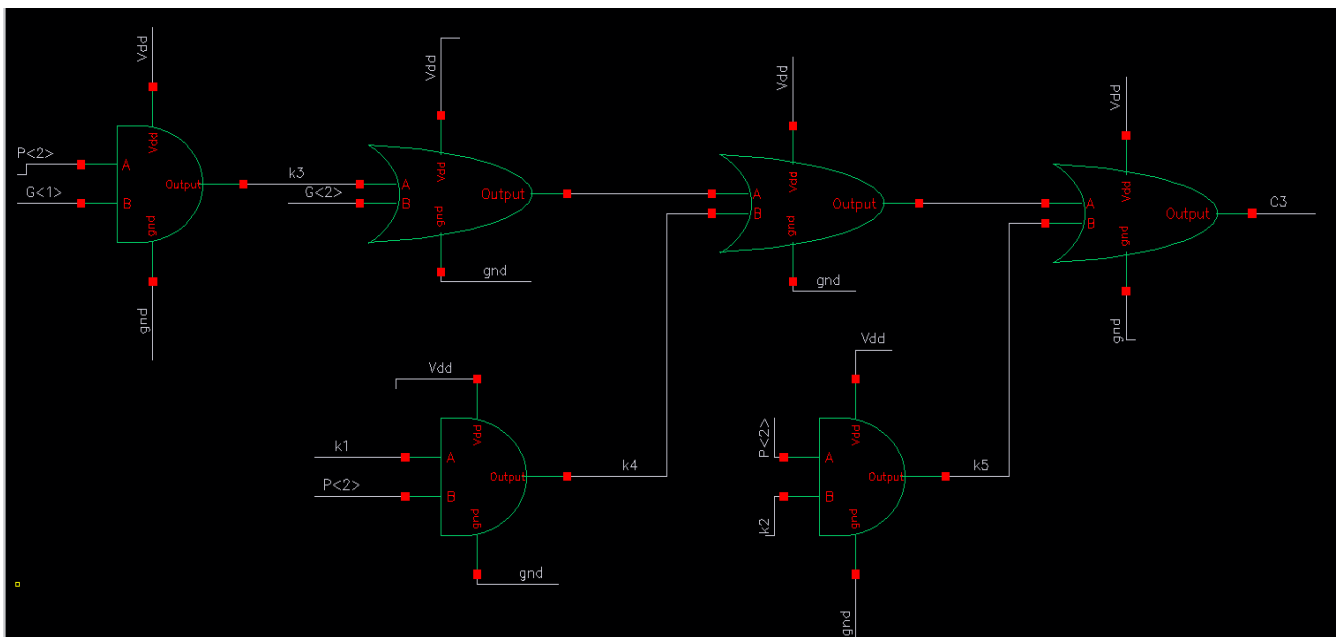
The schematic constructs the Boolean logic for the third carry bit. While the physical implementation uses a cascaded structure to evaluate all conditions from the initial **C0** to minimize delay, the fundamental logic dictates that a carry is generated at bit 3 if it is locally generated at bit 2, OR if a carry from the previous stage (**C2**) is successfully propagated through bit 2.

- **AND Logic:** The AND gates compute the propagation conditions. Ultimately, this represents the propagation of the previous stage's carry across the current bit ($P[2] \cdot C2$).
- **OR Logic:** A 4-input OR gate collects all the expanded conditions in the schematic. Logically, this equates to taking $G[2]$ directly, along with the propagated carry from the previous stage. If either condition is true, **C3** is driven HIGH.

Boolean Equation

The standard unexpanded formula for **C3** is:

$$C3 = G2 + P2 \cdot G1 + P2 \cdot P1 \cdot G0 + P2 \cdot P1 \cdot P0 \cdot C0$$



Carry 3 Generation Block

3.3.2.4 [[Carry 4 (C4) Generation Logic]]

This section computes the fourth and final carry-out signal, **C4**. Building upon the carry-lookahead principle, **C4** is calculated directly from the initial carry-in (**C0**) and the generate/propagate signals of all four bit positions, providing the final carry-out for the entire 4-bit adder block without waiting for intermediate ripple carries.

Interface Signals

Inputs:

- **C0**: The initial Carry-in signal.
- **P[0], P[1], P[2], P[3]**: Propagate signals from the 0th to 3rd CLA_Bit blocks.
- **G[0], G[1], G[2], G[3]**: Generate signals from the 0th to 3rd CLA_Bit blocks.

Output:

- **C4**: The final computed carry-out signal for the 4-bit CLA adder.

Logic Design and Architecture

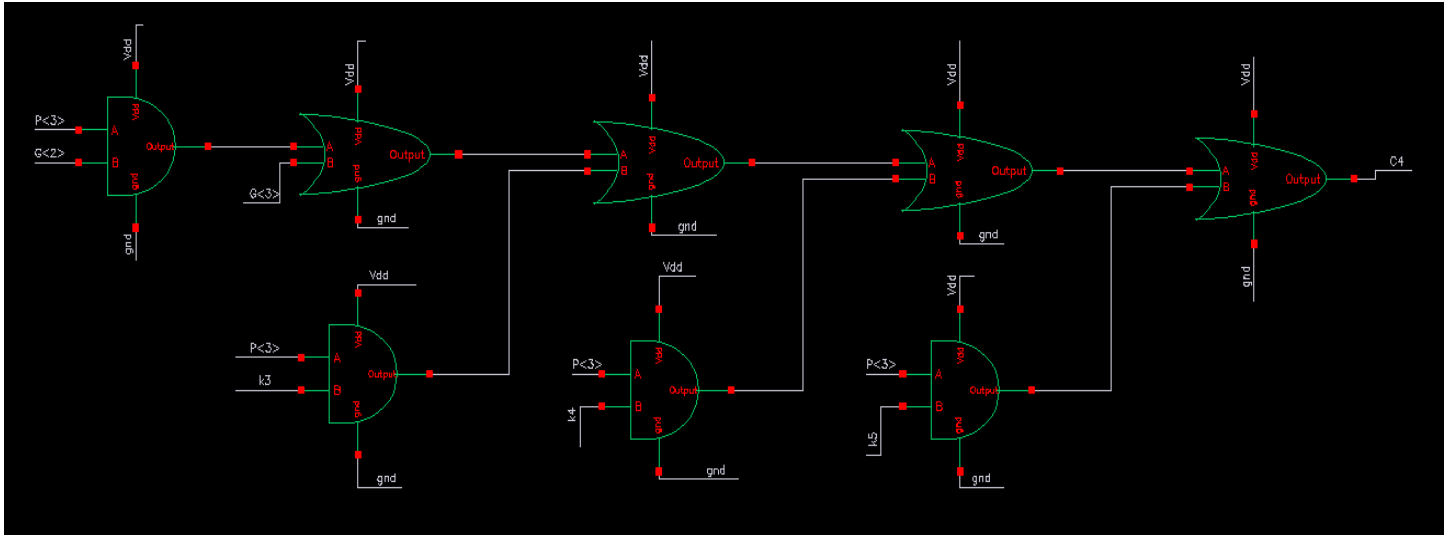
The schematic constructs the Boolean logic for the fourth carry bit. While the physical implementation utilizes a wider 5-input **OR** gate and a deeper cascaded structure of **AND** gates to evaluate all conditions directly from the initial **C0** (to ensure maximum parallel speed), the fundamental logic dictates that a carry is outputted if it is locally generated at bit 3, OR if a carry from the previous stage (**C3**) is successfully propagated through bit 3.

- **AND Logic**: Logically, the AND condition represents the propagation of the previous stage's carry across the highest bit ($P[3] \cdot C3$). The schematic physically achieves this by cascading multiple AND gates to compute all the individual propagation paths simultaneously.
- **OR Logic**: A 5-input OR gate collects all the expanded conditions shown in the schematic. Logically, this equates to taking the localized generate signal **G[3]** directly, along with the propagated carry from the previous stage. If either condition is true, **C4** is driven HIGH.

Boolean Equation

The standard unexpanded formula for **C4** is:

$$C4 = G3 + P3 \cdot G2 + P3 \cdot P2 \cdot G1 + P3 \cdot P2 \cdot P1 \cdot G0 + P3 \cdot P2 \cdot P1 \cdot P0 \cdot C0$$



Carry 4 Generation Block

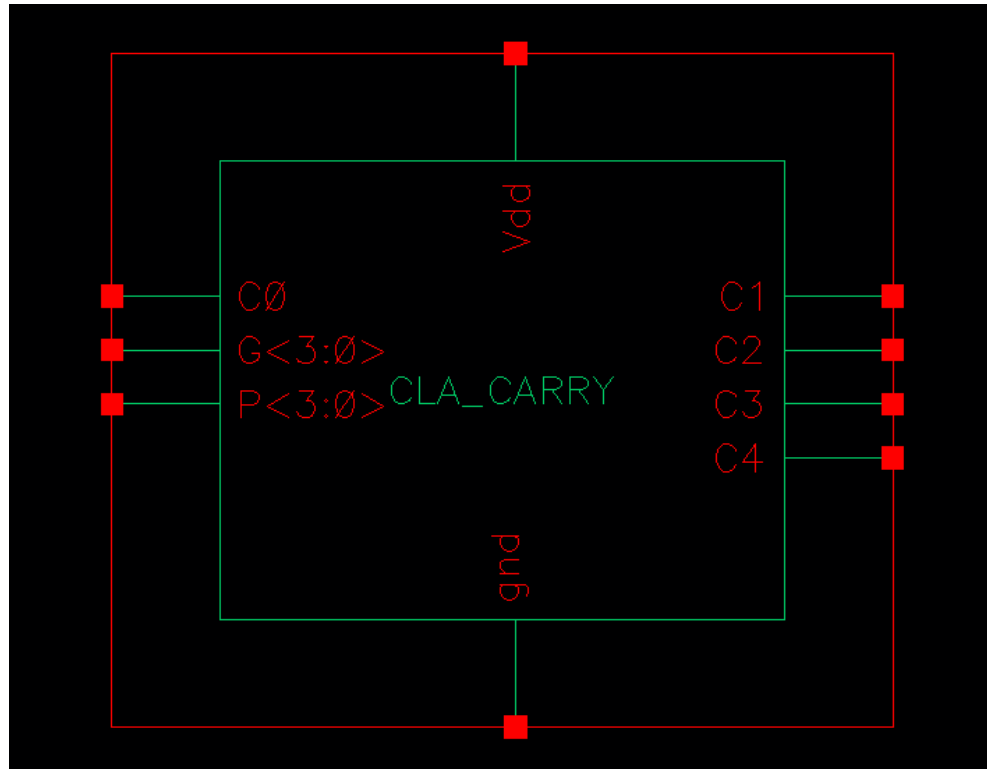
3.3.2.5 [[Final CLA CARRY Block]]

This final schematic represents the encapsulated symbol for the entire CLA_CARRY unit. This view highlights the component's role in the broader hierarchical design of the overall system.

Key Architectural Features

- **Design Abstraction:** This symbol hides the complex, cascaded gate-level routing discussed in the previous sections. By packaging the raw logic into a single, cohesive component, it ensures the top-level 4-bit adder schematic remains clean, readable, and easy to troubleshoot.
- **Bus Implementation:** To drastically reduce wiring clutter at the top level, the individual generates and propagates signals from the four bit-stages, which are grouped into consolidated 4-bit input buses ($G[3:0]$ and $P[3:0]$).

- **Signed Logic Integration:** Beyond standard carry generation, the unit incorporates an XOR gate ($P3 \oplus C4$) to detect overflows and establish the correct sign for 2's complement arithmetic, transforming the block into a signed-compatible component.



CLA_CARRY Final Block

3.3.3 [Top-Level 4-Bit CLA Architecture]

This schematic illustrates the complete integration of the 4-bit Carry Lookahead Adder. It brings together the individual bit-calculation stages and the centralized carry-generation engine to form the final operational circuit.

Interface Signals

Inputs:

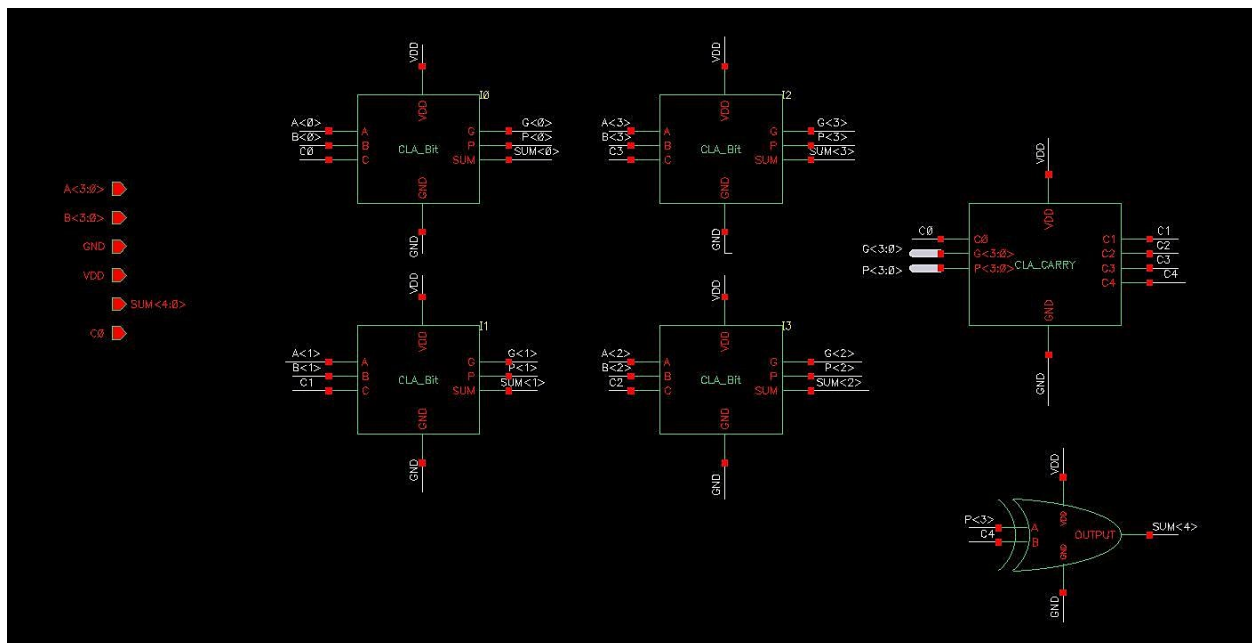
- **$A[3:0]$, $B[3:0]$:** The two 4-bit operands to be added.
- **$C0$:** The initial Carry-in signal.
- **VDD / GND :** Standard power supply and ground connections.

Outputs:

- **Sum[4:0]:** The final 5-bit signed result. The lower four bits (Sum[3:0]) are produced by the individual CLA_Bit blocks, while the most significant bit (Sum[4]) is the dedicated sign/overflow bit generated by the XOR logic within the CLA_CARRY unit.

System Integration

- **Bit Stages (CLA_Bit):** Four identical instances process the individual operand bits. They compute the local sum components while simultaneously outputting their respective Generate and Propagate signals.
- **Lookahead Engine (CLA_CARRY):** This block acts as the central processing hub for all carry logic. It receives the aggregated $G[3:0]$ and $P[3:0]$ buses, computes all intermediate carry signals ($C1$, $C2$, $C3$) in parallel, and feeds them directly back into the corresponding CLA_Bit stages to instantaneously finalize their sum calculations. Additionally, it processes the final overflow condition to ensure the 5th bit correctly represents the signed summation result.



4-Bit CLA Block

3.4 Full Perceptron

What is the Perceptron?

The perceptron is the complete system that ties everything together. It is a high-speed (1 GHz) signed artificial neuron that:

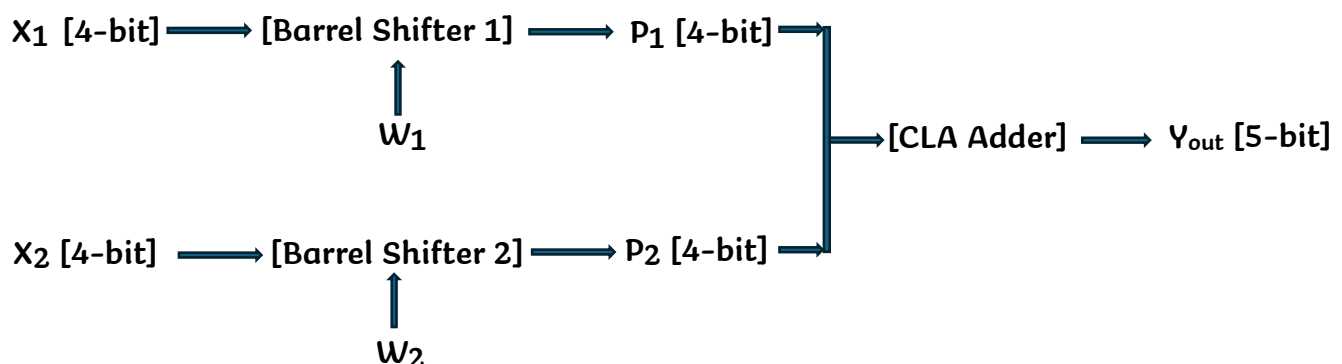
- Receives two 4-bit signed inputs X_1 and X_2 via Arithmetic Barrel Shifters.
- Applies programmable weights W_1 and W_2 to each input
- Accumulates the weighted using a 4-bit (CLA) with Signed Logic Integration.

Mathematically: $Y_{out} = \sum_i X_i W_i = X_1 \cdot W_1 + X_2 \cdot W_2$

Inputs and Outputs:

Signal	Width	Description
X_1	4-bit	First signed input (two's complement)
X_2	4-bit	Second signed input (two's complement)
W_1	2-bit	Weight for X_1 (controls Barrel Shifter 1)
W_2	2-bit	Weight for X_2 (controls Barrel Shifter 2)
Y_{out}	5-bit	Final signed sum. The MSB acts as the sign/decision bit.

System Architecture:



3.4.1 [The two inputs X_1 and X_2 arrive simultaneously]

The moment the circuit receives X_1 and X_2 , both are 4-bit signed numbers in two's complement form. They can range from:

- Most positive: $0111 = +7$
- Most negative: $1000 = -8$

Both inputs are generated randomly at 1 GHz, meaning a new pair of values arrives every 1 nanosecond. This is an extremely tight timing budget, which is exactly why every block in the design was optimized for speed.

At this point, nothing has happened yet — the inputs are simply sitting at the input ports of both barrel shifters, waiting for the weight signals.

3.4.2 [The weight signals W_1 and W_2 arrive]

Simultaneously with X_1 and X_2 , the 2-bit weight signals

$W_1 = \{S_1^1, S_0^1\}$ and $W_2 = \{S_1^2, S_0^2\}$ arrive at the control inputs of the two barrel shifters.

These weight signals determine how much each input will be scaled

The two weight signals are completely independent — W_1 can be 00 while W_2 is 11, or any other combination. There are 16 possible weight combinations in total (4×4), and the design was verified to work correctly for all of them.

$W (S_1 S_0)$	Shift Amount	Multiplication Factor
00	0 bits	$\times 1$
01	1 bit right	$\times 0.5$
10	2 bits right	$\times 0.25$
11	3 bits right	$\times 0.125$

3.4.3 [Barrel Shifter 1 processes X_1 with W_1]

X_1 enters Barrel Shifter 1 and gets arithmetically right-shifted according to W_1 .

Inside Barrel Shifter 1, the two-stage MUX structure activates:

- Stage 1 (controlled by S_0^1): decides shift of 0 or 1 bit
- Stage 2 (controlled by S_1^1): decides additional shift of 0 or 2 bits

The sign bit $X_1[3]$ is always preserved — it gets copied into any vacated MSB positions to maintain the correct sign of the result.

The output of Barrel Shifter 1 is $P_1 = X_1 \times W_1$ (a 4-bit signed result).

3.4.4 [Barrel Shifter 2 processes X_2 with W_2]

Completely in parallel with Step 3, X_2 enters Barrel Shifter 2 and undergoes the exact same process with W_2 as the control.

This parallelism is a key architectural advantage — both multiplications happen simultaneously, not sequentially, so the total delay of the multiplication stage is just one barrel shifter delay, not two.

The output of Barrel Shifter 2 is $P_2 = X_2 \times W_2$ (another 4-bit signed result).

3.4.5 [P_1 and P_2 enter the Carry Lookahead Adder]

Now both 4-bit weighted results P_1 and P_2 arrive at the inputs of the Carry Lookahead Adder (CLA). The adder's job is to compute: $Y_{\text{out}} = P_1 + P_2$

Inside the CLA, the following happens simultaneously for all 4 bits:

3.4.5.1 [[Generate and Propagate signals are computed]]

For each bit position i , the CLA_bit cell computes:

- $G_i = P_{1i} \cdot P_{2i} \rightarrow$ this bit position will generate a carry regardless of carry-in
- $P_i = P_{1i} \oplus P_{2i} \rightarrow$ this bit position will propagate a carry if one arrives

3.4.5.2 [[All carry signals computed simultaneously by CLA_logic]]

Unlike a ripple carry adder where C_1 must finish before C_2 can start, the CLA computes all carries at the same time:

- $C_1 = G_0 + P_0 \cdot C_0$
- $C_2 = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0$
- $C_3 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0$
- $C_4 = G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0$

Since $C_0 = 0$ (no initial carry-in), these simplify slightly but all still resolve in one logic level delay rather than propagating one by one.

3.4.5.3 [[Sum bits are computed]]

With all carries now available simultaneously, each sum bit is computed as:

- $S_i = P_i \oplus C_i$

All 4 sum bits resolve at virtually the same time.

3.4.6 [Sign extension to 5-bit output]

The system incorporates a Signed Logic Integration within the CLA unit to handle 2's complement arithmetic correctly.

- **The Logic:** The 5th bit (Sum[4]) is generated by an **XOR gate** that processes the last propagate signal (P_3) and the final carry (C_4).

This specific logic ensures that the 5th bit acts as a true Sign/Overflow bit.

It guarantees that the output correctly represents the result in the 2's complement range (from -16 to +15), preventing the errors that usually occur in simple unsigned adders when dealing with negative values.

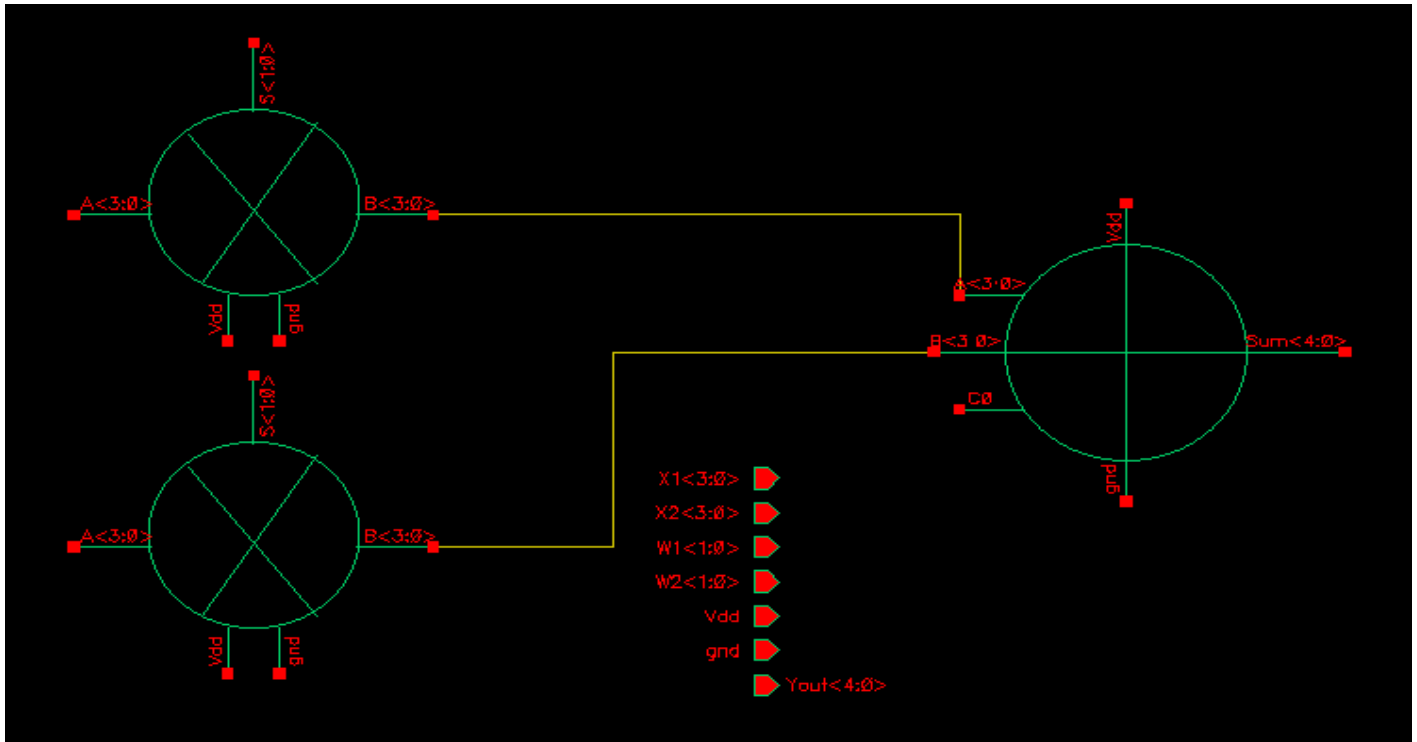
The final 5-bit output Y_{out} is now ready.

3.4.7 [Output is stable and valid]

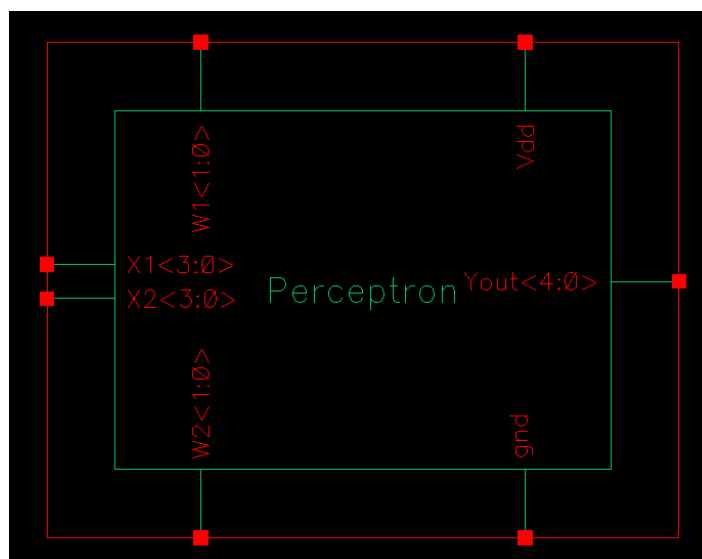
After all the propagation delays through the barrel shifters and the CLA adder have settled, Y_{out} holds the correct signed result of $X_1 \cdot W_1 + X_2 \cdot W_2$.

The next clock edge (1 ns later) brings new values of X_1 , X_2 , W_1 , W_2 and the entire process repeats itself.

Full Perceptron



schematic



symbol

4. Simulation Results and Functional Verification

To ensure that the architecture is valid and reliable when operated at high frequencies, we performed a comprehensive test simulation. We verify that our Perceptron works properly in all cases for different kinds of inputs encoded in 2's complement notation. The critical point here is to make sure that our Arithmetic Barrel Shifters and Signed CLA components can operate harmoniously to produce the right 5-bit weighted sum.

4.1 4-bit Barrel Shifter (Multiplier) simulation

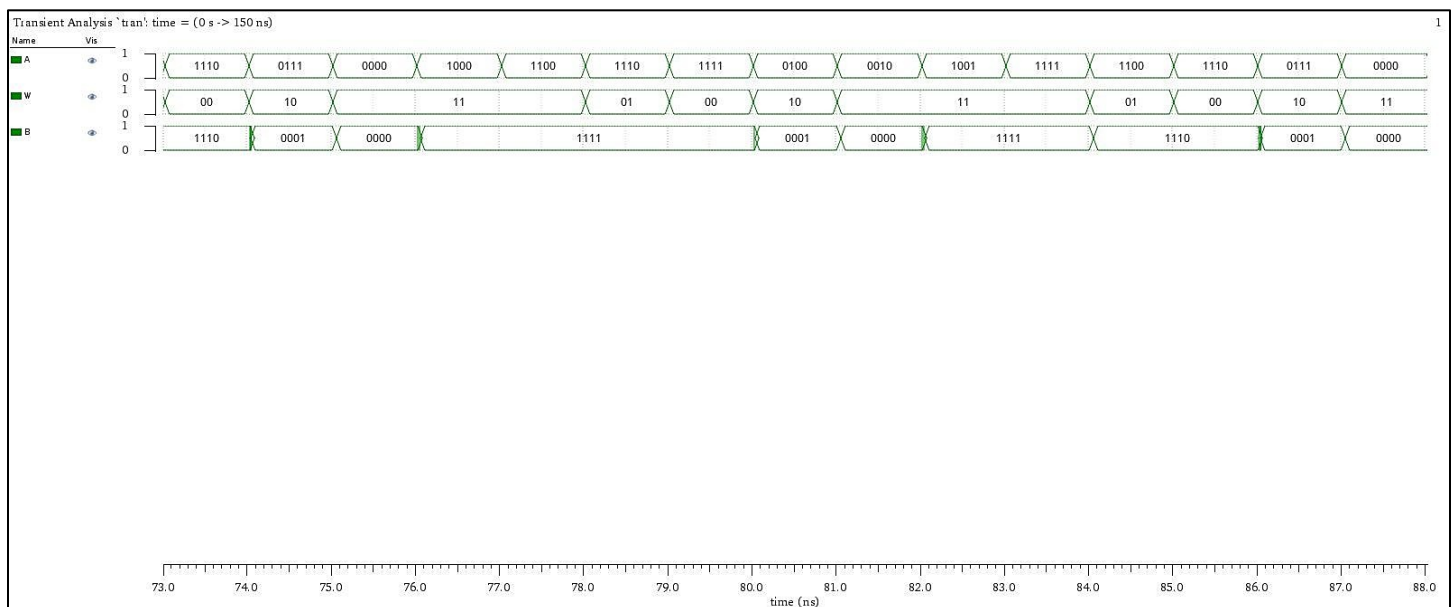


Figure 1: Transient Analysis of the Arithmetic Barrel Shifter (Multiplication Stage)

From the waveform generated through the simulation, the arithmetic barrel shifter operates as required using a clock speed of 1 GHz. Both positive and negative values of 4-bits (A) are arithmetically shifted according to the value of the two bits (W). It is worth noting that the performance of ASR on negative values yields an output that retains its sign bit (B). For instance, a value of 1110 at 73 ns and 1000 at 76 ns are correctly retained in the outputs. The output steps are well defined and occur in a time period of one nanosecond.

4.2 Carry Lookahead Adder (CLA) simulation

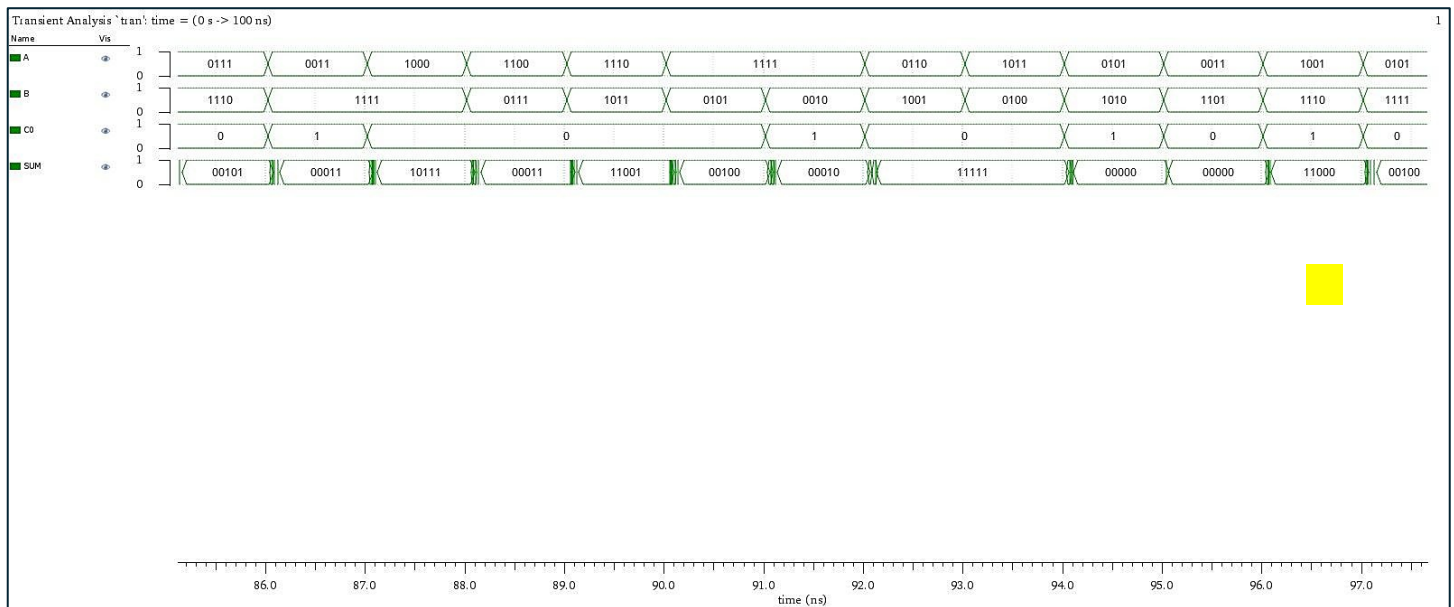


Figure 2: Transient simulation results for the 4-bit Signed CLA at 1 GHz

The figure demonstrates accurate 2's complement summation. The waveforms show the transition of inputs A and B, the carry-in (C0), and the resulting 5-bit signed output (SUM), ensuring functional correctness and timing stability within the 1 ns clock period.

4.3 Full Perceptron System Integration simulation

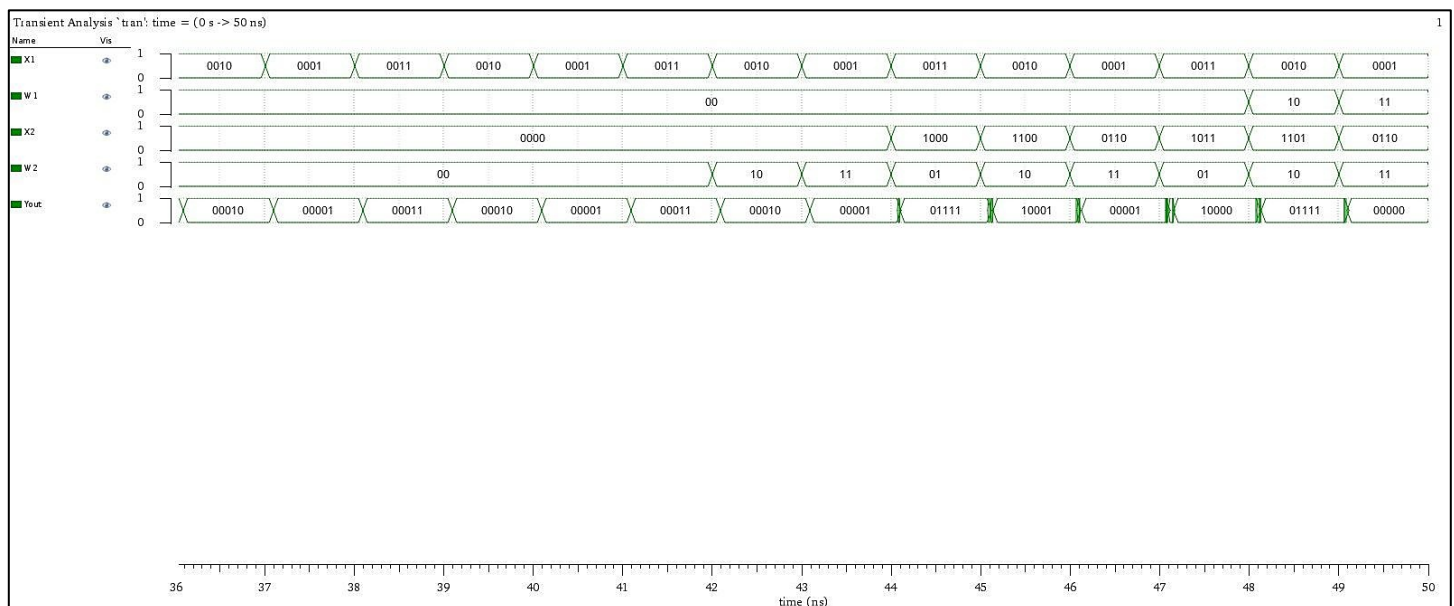


Figure 3: Transient Analysis of the complete Signed Perceptron at 1 GHz

This waveform depicts the actual operation of the dual arithmetic barrel shifter in conjunction with the carry-lookahead adder. From the simulation, the 5-bit output, Y_{out} , is generated from various sets of vectors and remains completely synchronized to both the weighted shifting process and the parallel addition process. It can be seen from the above that the system performs the task of implementing 2's complement arithmetic successfully throughout the whole data path, producing an accurate output every 1 ns clock cycle, thereby achieving the desired 1 GHz performance rate.

5. Hand Analysis & Simulation Results For Logic Gates Delay

The theoretical analysis of digital CMOS circuits provides a fundamental baseline for evaluating design correctness and performance. While Cadence Virtuoso simulation captures the full complexity of transistor-level behavior including parasitic capacitances, short-channel effects, and non-ideal switching hand analysis based on ideal CMOS models offers a clean and interpretable reference point for verifying that the design operates as intended.

In this section, each building block of the perceptron is analyzed by hand using standard CMOS delay and power models, and the results are compared against the post-simulation measurements extracted from Cadence. The analysis covers the fundamental logic gates: INVERTER, NAND, NOR, AND, OR, XOR.

5.1 Device Parameters and Capacitance Extraction

<u>FOR NMOS TRANSISTOR</u>	<u>FOR PMOS TRANSISTOR</u>
$Req = 6.4148 \text{ k}\Omega$	$Req = 6.0852 \text{ k}\Omega$
$C_{dn} = 0.2803 \text{ fF}$	$C_{dp} = 0.5413 \text{ fF}$
$C_{gn} = 0.1203 \text{ fF}$	$C_{gp} = 0.2244 \text{ fF}$
$C_l = 1 \text{ fF}$	
$C_{gn} + C_{gp} = 0.3447 \text{ fF}$	
$C_{dn} + C_{dp} = 0.8216 \text{ fF}$	

5.2 CMOS inverter delay

$$\begin{aligned}
 t_{p,inv} &= 0.69 \times R_{avg} \times (C_{dn} + C_{dp} + C_l) \\
 &= 0.69 \times 6.25 \times 10^3 \times (28.03 \times 10^{-17} + 54.13 \times 10^{-17} + 10^{-15}) \\
 &= 7.85565 \text{ ps}
 \end{aligned}$$

$$t_{p,int,inv} = 0.69 \times R_{avg} \times (C_{dn} + C_{dp}) = 3.54315 \text{ ps}$$

$$t_{p,LH} = 3.05466 \text{ ps} \quad \text{,,,} \quad t_{p,HL} = 3.20871 \text{ ps} \quad \text{,,,} \quad t_p = \frac{t_{p,LH} + t_{p,HL}}{2} = 3.131685 \text{ ps}$$

$$error = \frac{3.54315 - 3.131685}{3.131685} \times 100 = 13.138 \%$$

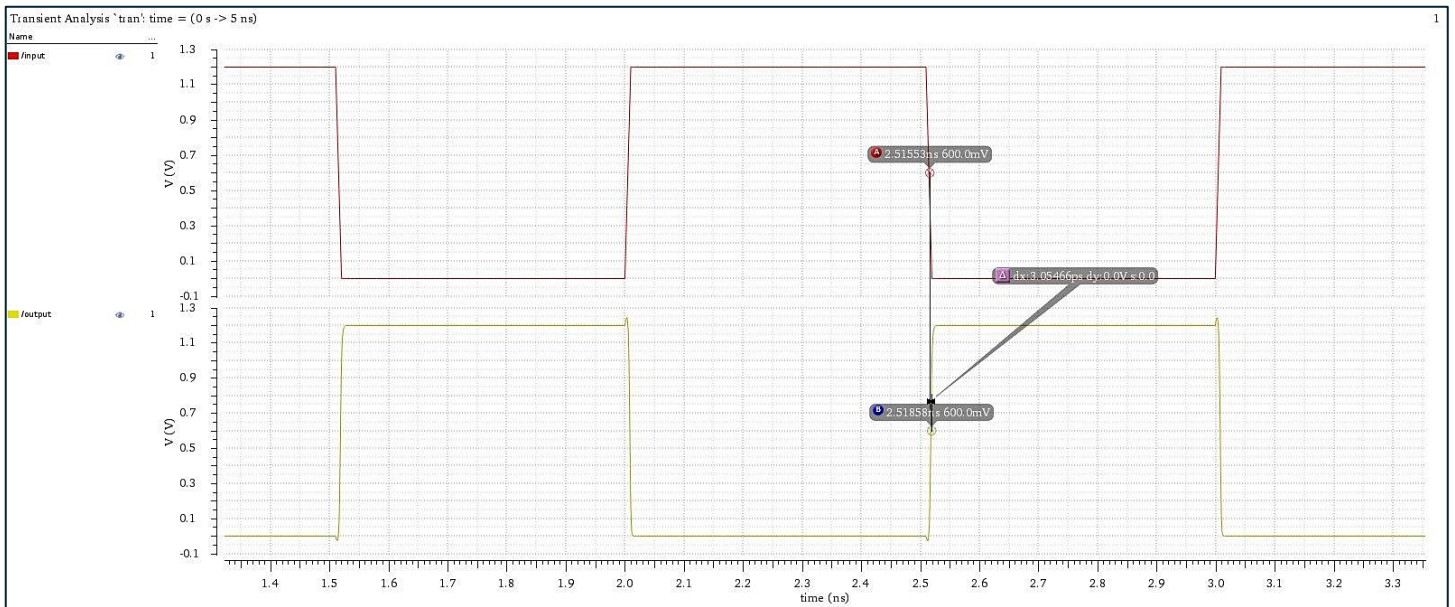


Figure 4: Transient simulation in Cadence showing a measured t_{pLH} for NAND gate of 13.175 ps at the 50% voltage threshold

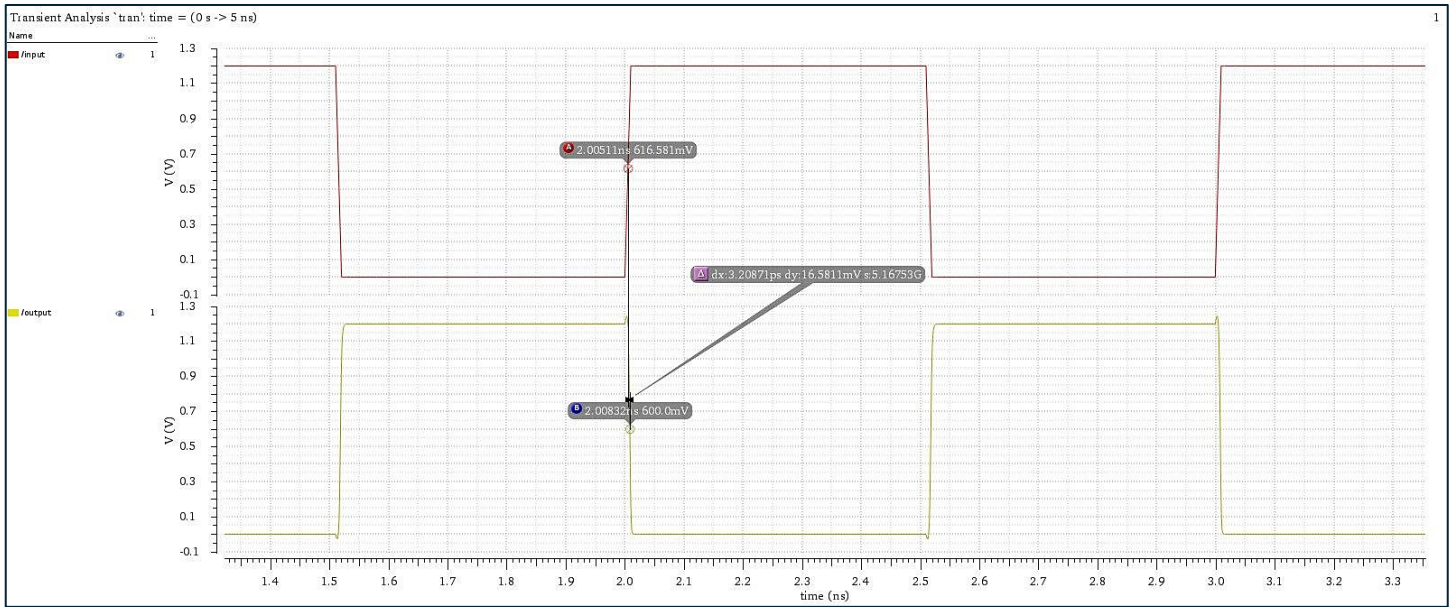


Figure 5: Transient simulation in Cadence showing a measured t_{pHL} for NAND gate of 8.9936 ps at the 50% voltage threshold

5.3 NAND gate delay analysis

ASSUME: $R_{eqn} = R_{eqp} = R_{avg} = 6.25 \text{ k}\Omega$

$$\begin{aligned}
 t_{p,hand} &= 0.69 \times R_{avg} \times (2C_{dn} + 2C_{dp} + C_l) \\
 &= 0.69 \times 6.25 \times 10^3 \left((2 \times 28.03 + 2 \times 54.13) \times 10^{-17} + 10^{-15} \right) \\
 &= 11.3988 \text{ ps}
 \end{aligned}$$

$$t_{p,LH} = 13.175 \text{ ps} \quad , \quad t_{p,HL} = 8.9936 \text{ ps} \quad , \quad t_p = \frac{t_{p,LH} + t_{p,HL}}{2} = 11.0843 \text{ ps}$$

$$error = \frac{11.3988 - 11.0843}{11.0843} \times 100 = 2.837 \%$$

$$t_{p,int,NAND} = 0.69 \times R_{avg} \times (2C_{dn} + 2C_{dp}) = 7.0863 \text{ ps}$$

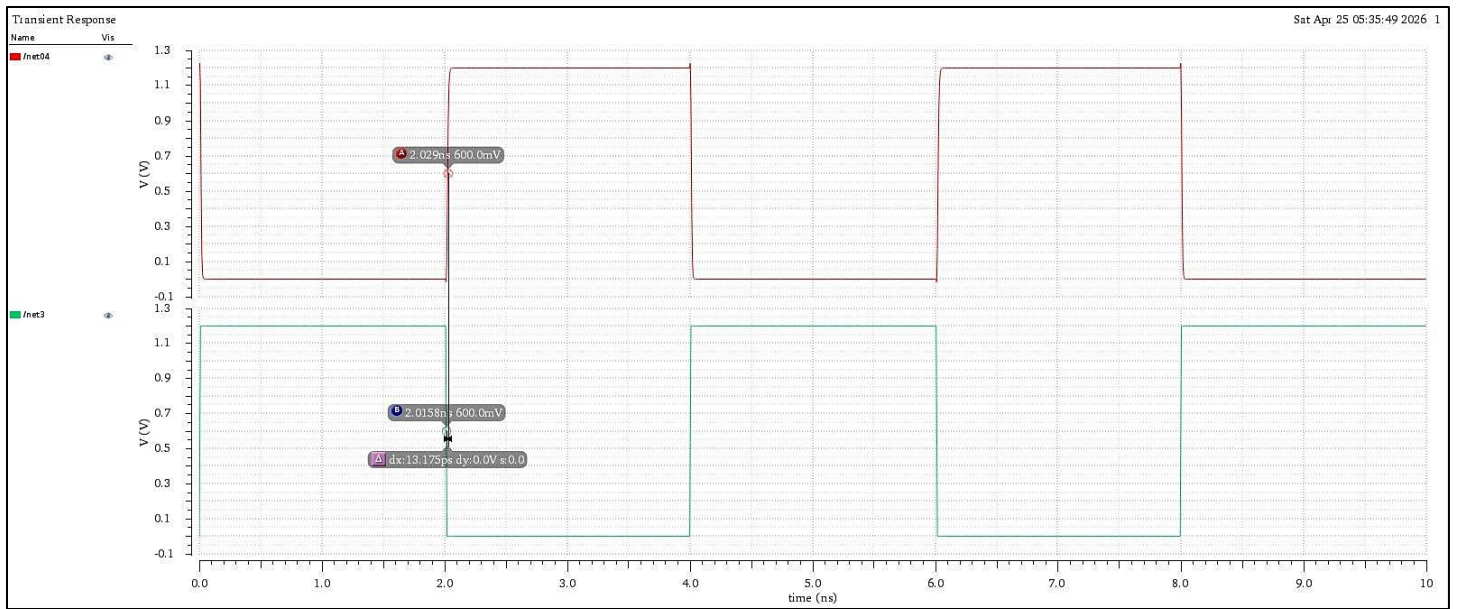


Figure 6: Transient simulation in Cadence showing a measured t_{pLH} for NAND gate of 13.175 ps at the 50% voltage threshold.

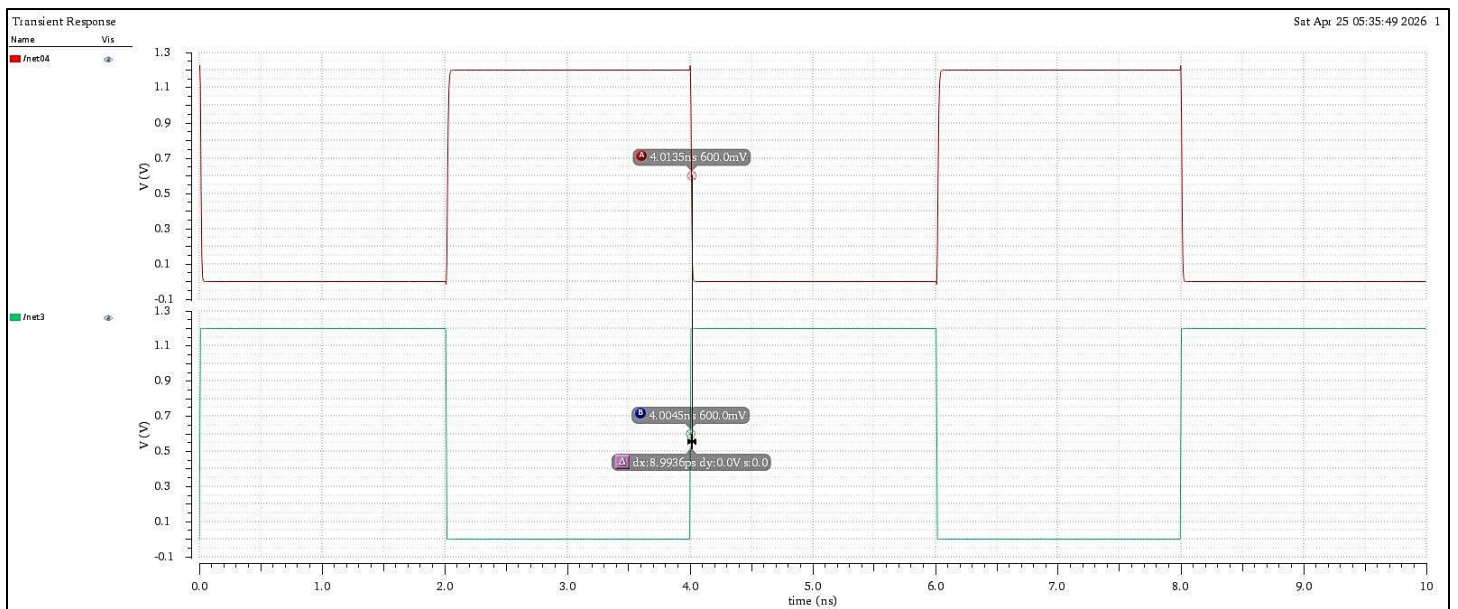


Figure 7: Transient simulation in Cadence showing a measured t_{pHL} for NAND gate of 8.9936 ps at the 50% voltage threshold.

5.4 AND gate delay analysis

$$\begin{aligned}t_{p,hand} &= 0.69 \times R_{avg} \times (2 \times C_{dn} + 2 \times C_{dp} + C_{gp} + C_{gn}) \\&\quad + 0.69 \times R_{avg} \times (C_{dp} + C_{dn} + C_l) \\&= 0.69 \times 6.25 \times 10^3 [(2 \times 28.03 + 2 \times 54.13) \times 10^{-17} + (120.3 + 224.4) \times 10^{-18}] \\&\quad + 0.69 \times 6.25 \times 10^3 [(54.13 + 28.03) \times 10^{-17} + 10^{-15}] \\&= 16.428 \text{ ps}\end{aligned}$$

$$t_{p,LH} = 15.207 \text{ ps} \quad , \quad t_{p,HL} = 17.889 \text{ ps} \quad , \quad t_p = \frac{t_{p,LH} + t_{p,HL}}{2} = 16.548 \text{ ps}$$

$$error = \left| \frac{16.428 - 16.548}{16.548} \right| \times 100 = 0.725 \%$$

$$\begin{aligned}t_{p,int,AND} &= 0.69 \times R_{avg} \times (2 \times C_{dn} + 2 \times C_{dp} + C_{gp} + C_{gn}) \\&\quad + 0.69 \times R_{avg} \times (C_{dp} + C_{dn}) \\&= 0.69 \times 6.25 \times 10^3 [(2 \times 28.03 + 2 \times 54.13) \times 10^{-17} + (120.3 + 224.4) \times 10^{-18}] \\&\quad + 0.69 \times 6.25 \times 10^3 [(54.13 + 28.03) \times 10^{-17}] \\&= 12.11596875 \text{ ps}\end{aligned}$$

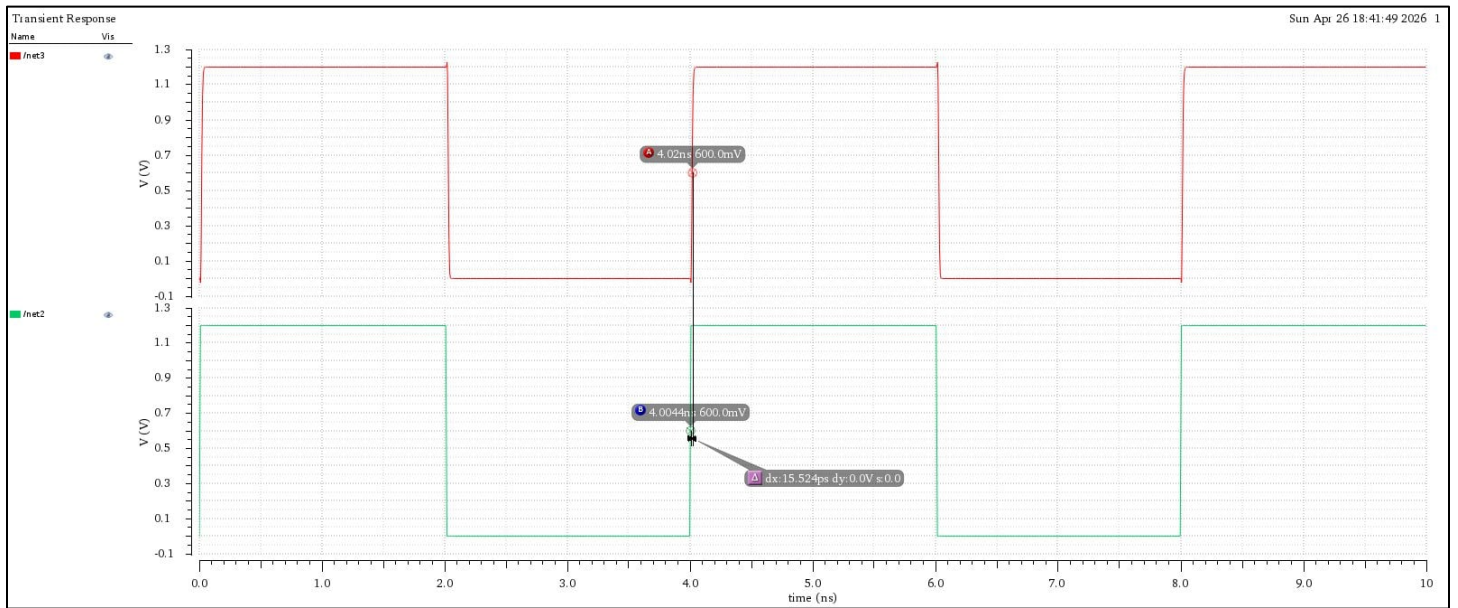


Figure 8: Transient simulation in Cadence showing a measured t_{pLH} for AND gate of 15.524 ps at the 50% voltage threshold

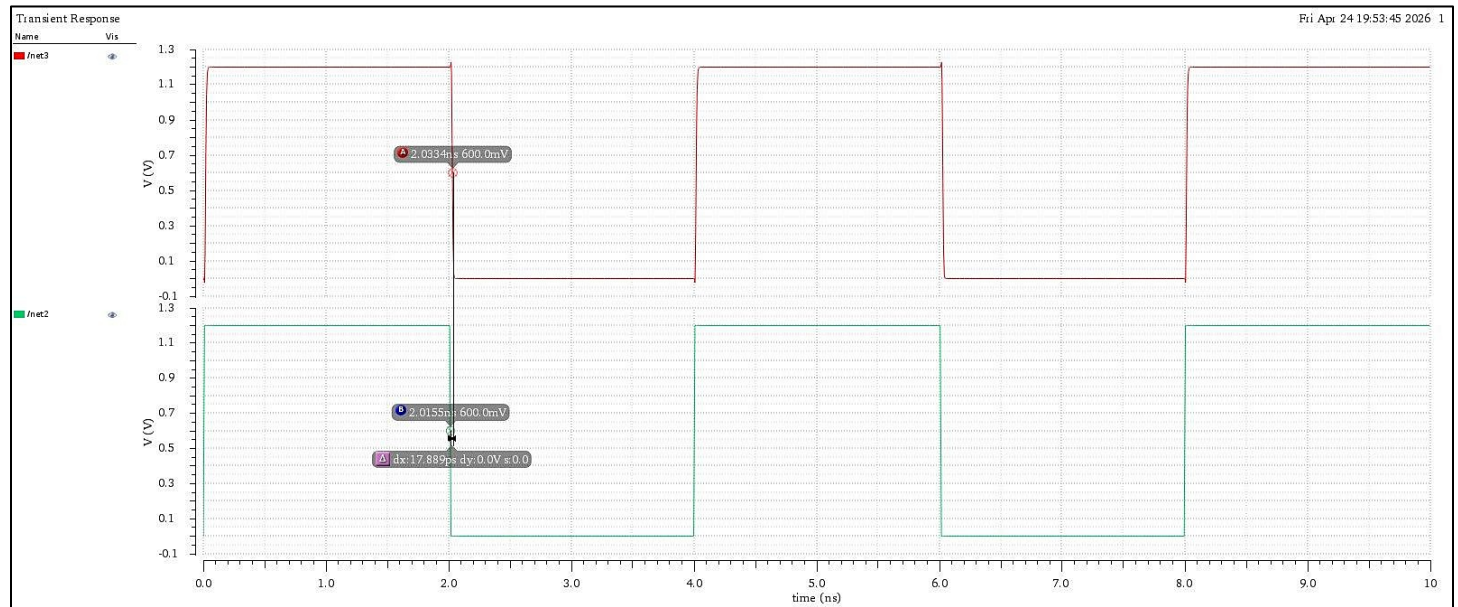


Figure 6: Transient simulation in Cadence showing a measured t_{pHL} for AND gate of 17.889 ps at the 50% voltage threshold

5.5 OR gate delay analysis

$$\begin{aligned}t_{p,hand} &= 0.69 \times R_{avg} \times (2C_{dn} + 2C_{dp} + C_{gn} + C_{gp}) + t_{p,inv} \\&= 0.69 \times R_{avg} \times (2C_{dn} + 2C_{dp} + C_{gn} + C_{gp}) + 0.69 \times R_{avg} \times (C_{dn} + C_{dp} + C_l) \\&= 0.69 \times 6.25 \times 10^3 \times (2 \times 28.03 \times 10^{-17} + 2 \times 54.13 \times 10^{-17} + 3.447 \times 10^{-16}) \\&\quad + 0.69 \times 6.25 \times 10^3 \times (28.03 \times 10^{-17} + 54.13 \times 10^{-17} + 10^{-15}) \\&= 16.428 \text{ ps}\end{aligned}$$

$$t_{p,LH} = 17.231 \text{ ps} \quad , , , \quad t_{p,HL} = 14.806 \text{ ps} \quad , , , \quad t_p = \frac{t_{p,LH} + t_{p,HL}}{2} = 16.0185 \text{ ps}$$

$$error = \left| \frac{16.428 - 16.0185}{16.0185} \right| \times 100 = 2.55642 \%$$

$$\begin{aligned}t_{p,int,OR} &= 0.69 \times R_{avg} \times (2C_{dn} + 2C_{dp} + C_{gn} + C_{gp}) + t_{p,inv} \\&= 0.69 \times R_{avg} \times (2C_{dn} + 2C_{dp} + C_{gn} + C_{gp}) + 0.69 \times R_{avg} \times (C_{dn} + C_{dp}) \\&= 0.69 \times 6.25 \times 10^3 \times (2 \times 28.03 \times 10^{-17} + 2 \times 54.13 \times 10^{-17} + 3.447 \times 10^{-16}) \\&\quad + 0.69 \times 6.25 \times 10^3 \times (28.03 \times 10^{-17} + 54.13 \times 10^{-17}) \\&= 12.11596875 \text{ ps}\end{aligned}$$

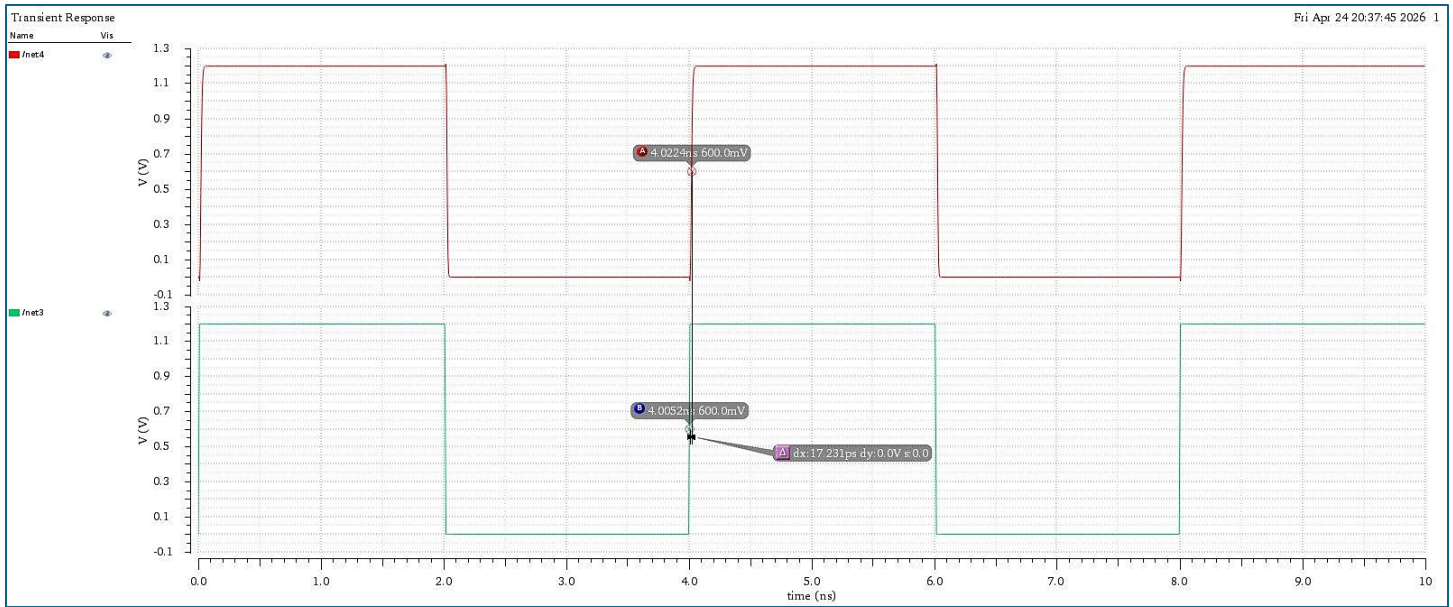


Figure 10: Transient simulation in Cadence showing a measured t_{pLH} for OR gate of 17.231 ps at the 50% voltage threshold.

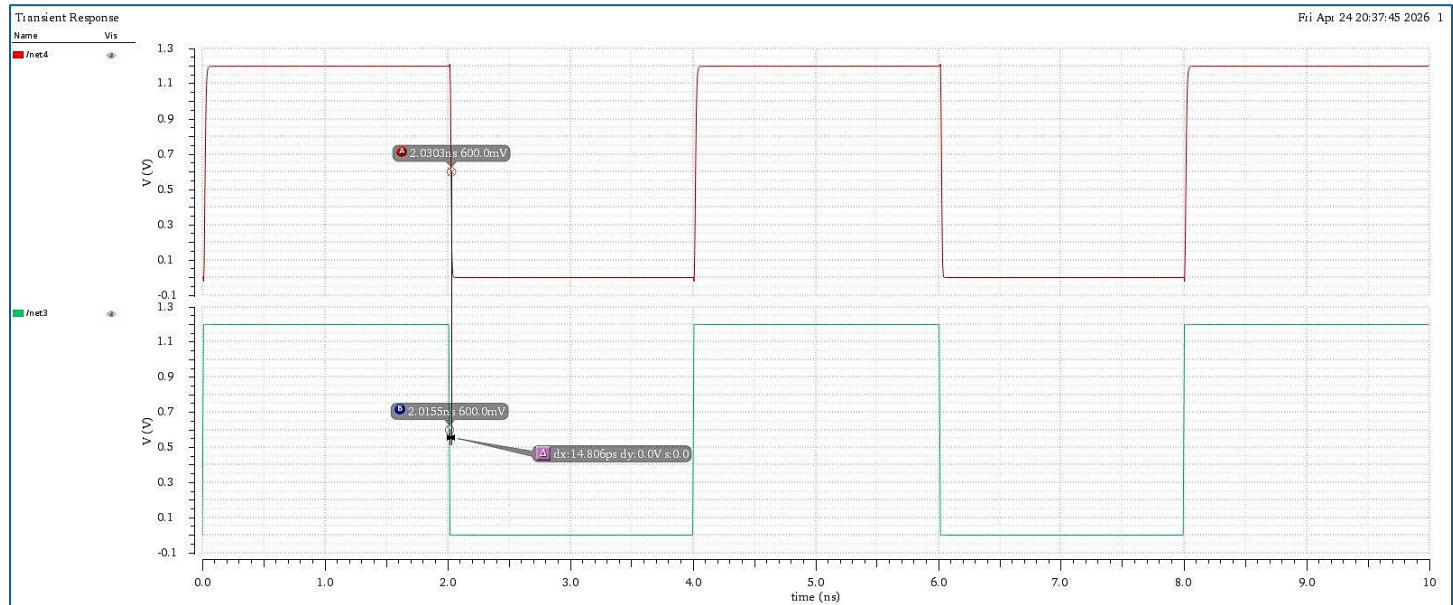
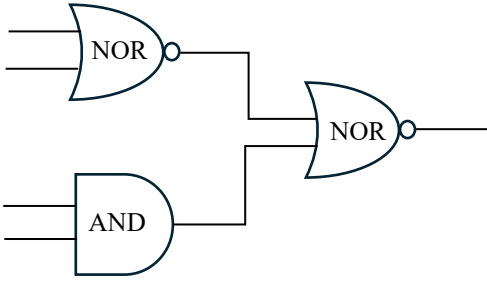


Figure 9: Transient simulation in Cadence showing a measured t_{pHL} for OR gate of 14.806 ps at the 50% voltage threshold

5.6 XOR gate delay analysis

$$t_{p,int,NAND} = t_{p,int,NOR}$$

$$\therefore t_{p,int,AND} > t_{p,int,NOR}$$



$$t_{p,XOR} = t_{p,AND} + t_{p,NOR}$$

$$t_{p,hand} = t_{p,NAND} + t_{p,inv} + t_{p,NOR}$$

$$\begin{aligned} t_{p,hand} &= 0.69 \times R_{avg} \times (2C_{dn} + 2C_{dp} + C_{gn} + C_{gp}) \\ &\quad + 0.69 \times R_{avg} \times (C_{dn} + C_{dp} + C_{gn} + C_{gp}) \\ &\quad + 0.69 \times R_{avg} \times (2C_{dn} + 2C_{dp} + C_l) \end{aligned}$$

$$\begin{aligned} t_{p,hand} &= 0.69 \times 6.25 \times 10^3 \times (2 \times 82.16 \times 10^{-17} + 3.447 \times 10^{-16}) \\ &\quad + 0.69 \times 6.25 \times 10^3 \times (82.16 \times 10^{-17} + 3.447 \times 10^{-16}) \\ &\quad + 0.69 \times 6.25 \times 10^3 \times (2 \times 82.16 \times 10^{-17} + 10^{-15}) \\ &= 25 \text{ ps} \end{aligned}$$

$$t_{p,LH} = 23.852 \text{ ps} \quad , \quad t_{p,HL} = 23.235 \text{ ps} \quad , \quad t_p = \frac{t_{p,LH} + t_{p,HL}}{2} = 23.5435 \text{ ps}$$

$$error = \left| \frac{25 - 23.5435}{23.5435} \right| \times 100 = 6.18642 \%$$

$$\begin{aligned}
t_{p,int,XOR} &= 0.69 \times R_{avg} \times (2C_{dn} + 2C_{dp} + C_{gn} + C_{gp}) \\
&\quad + 0.69 \times R_{avg} \times (C_{dn} + C_{dp} + C_{gn} + C_{gp}) \\
&\quad + 0.69 \times R_{avg} \times (2C_{dn} + 2C_{dp}) \\
t_{p,int,XOR} &= 0.69 \times 6.25 \times 10^3 \times (2 \times 82.16 \times 10^{-17} + 3.447 \times 10^{-16}) \\
&\quad + 0.69 \times 6.25 \times 10^3 \times (82.16 \times 10^{-17} + 3.447 \times 10^{-16}) \\
&\quad + 0.69 \times 6.25 \times 10^3 \times (2 \times 82.16 \times 10^{-17}) \\
&= 20.6875 \text{ ps}
\end{aligned}$$

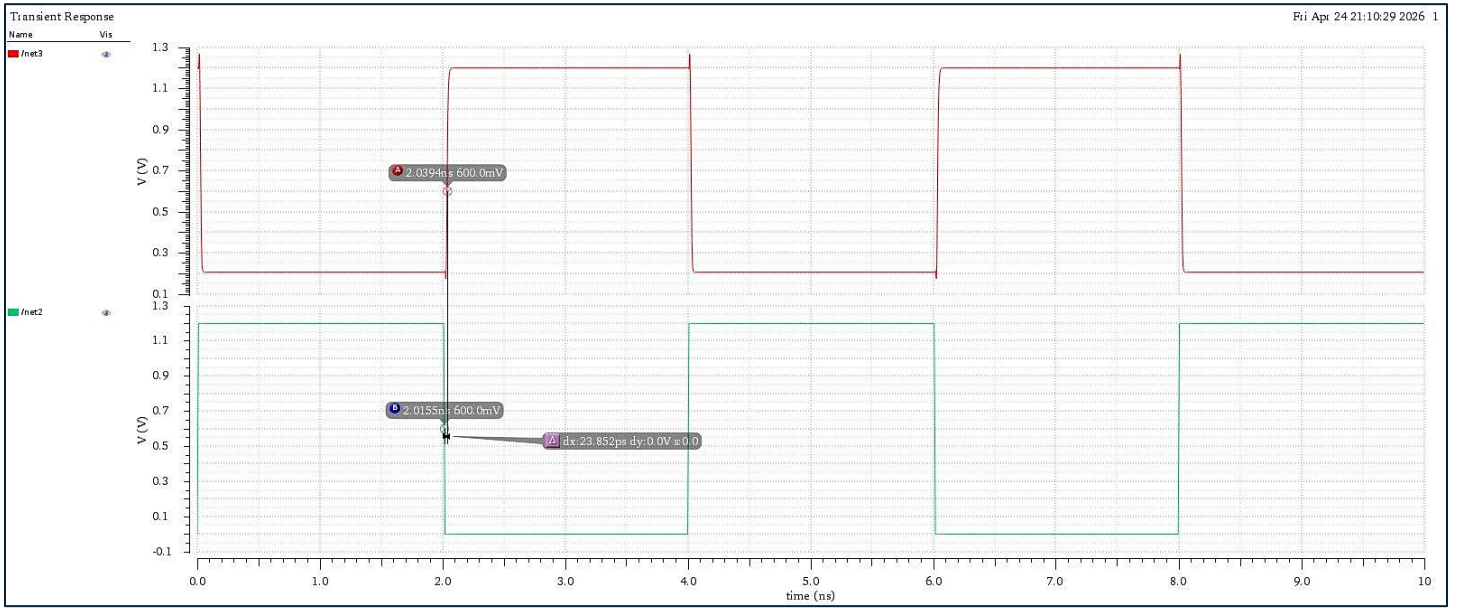


Figure 11: Transient simulation in Cadence showing a measured t_{pLH} for XOR gate of 23.852 ps at the 50% voltage threshold

6. Hand Analysis & Simulation Results For Perceptron Delay

In this section, the hand-calculated propagation delays for each building block of the perceptron are presented and compared against the corresponding Cadence Virtuoso simulation results including MUX, MULTIPLIER, and PERCEPTRON.

6.1 Isolation to a specific input-to-output path

The following values and conditions are given for this case:

$$\begin{array}{llll} X1 = 1\ 1\ 1\ 1 & ,,, & W1 = 0\ 0 & ,,, & W2 = 0\ 0 \\ X2<0> = 0 & ,,, & X2<1> = 0 & & \\ X2<2> \text{ transition from 0 to 1} & ,,, & X2<3> = 0 & & \end{array}$$

- Output of first multiplier: B1<3:0>
- Output of second multiplier: B2<3:0>
- Calculate the delay between: X2<2> and sum<4>
- This represents the transition from zero to 1 ($0 \rightarrow 1$)

6.2 The standard electrical parameters for every gate in the design

Standard cell parameters used throughout the delay calculations:

- 2:1 MUX

$$\begin{aligned} R_{mux} &= R_{inv} \\ C_{in}(mux) &= C_{in}(inverter) \end{aligned}$$

- Basic Gates

$$R(xor) = R_{in}(or) = R_{in}(and) = R_{in}(inv) = R_{avg} = R = 6.25\ K$$

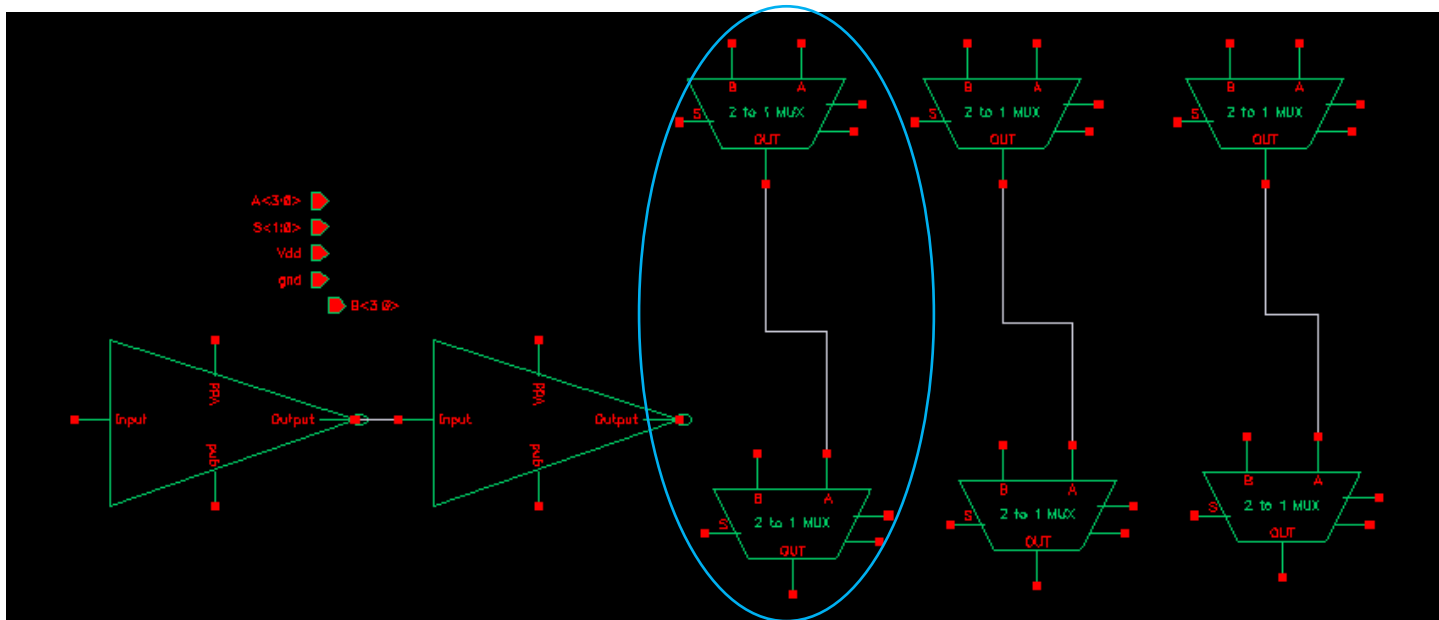
$$C_{gin,nor,or} = 0.5691\ fF \quad ,,, \quad C_{gin,nand,and} = 0.465\ fF$$

$$C_{gin,xor} = C_{gin,and} + C_{gin,nor} = 1.0341\ fF$$

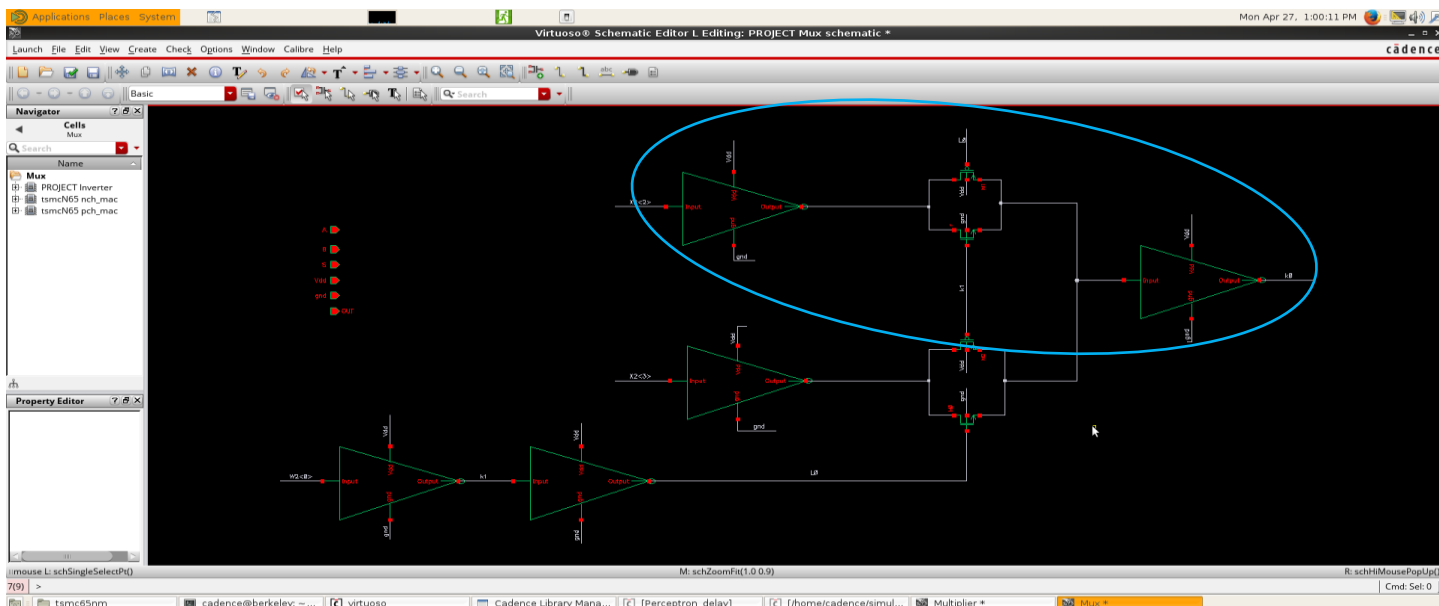
$$Cdd_{pmos+nmos} = 0.8216\ fF$$

6.3 Stage-by-Stage Delay Calculation for the Multiplier

➤ **Full stage : Two Muxes**

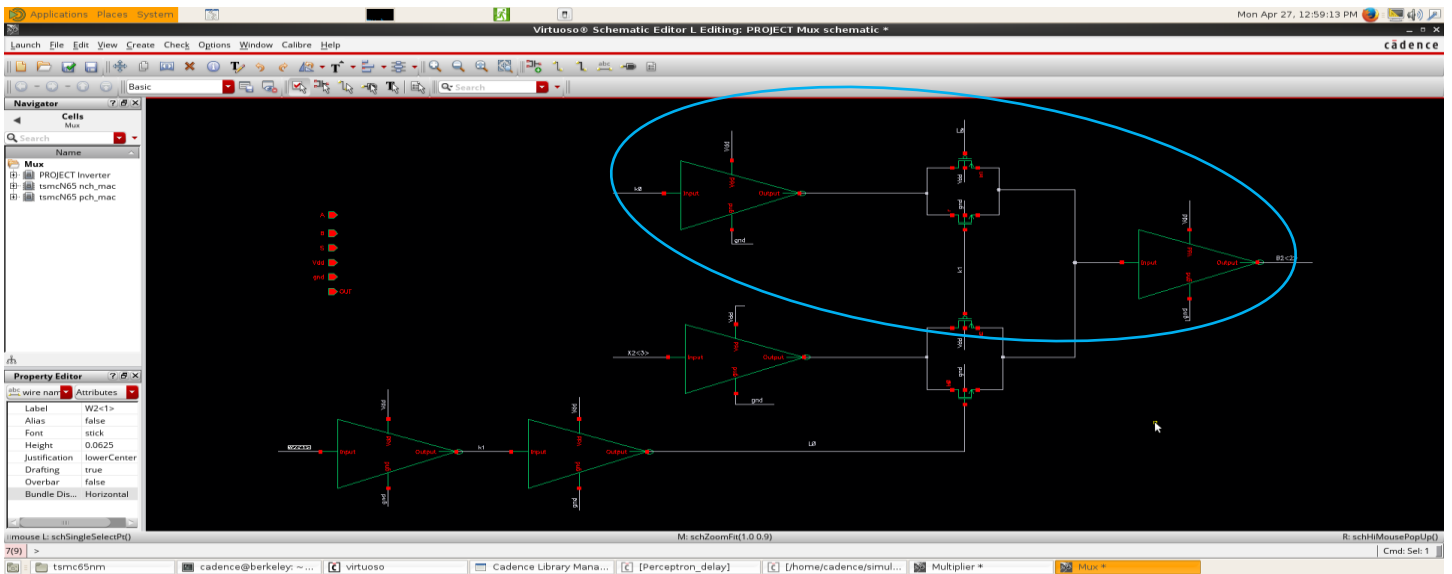


➤ **First Mux Stage**



$$\begin{aligned}
t_{p1} &= t_{p,inv+TG} + t_{p,inv} \\
&= 0.69 (R)(2Cdd_{pmos+nmos}) + 0.69(R // R) (2Cdd_{pmos+nmos} + Cgg) \\
&\quad + 0.69 R (2Cgg + Cdd_{pmos+nmos}) \\
t_{p1} &= 17.88889ps
\end{aligned}$$

➤ Second Mux Stage



$$\begin{aligned}
 t_{p2} &= t_{p,inv+TG} + t_{p,inv} \\
 &= 0.69(R)(2Cdd_{pmos+nmos}) + 0.69(R // R)(2Cdd_{pmos+nmos} + Cgg) \\
 &\quad + 0.69R(Cdd_{pmos+nmos} + C_{XOR,CLA} + C_{AND,CLA}) \\
 t_{p2} &= 21.38ps
 \end{aligned}$$

➤ Total Multiplier Delay

$$\begin{aligned}
 t_{multi} &= t_{p1} + t_{p2} \\
 &= 19.3653ps + 20.8553ps = 40.22ps \\
 \therefore t_{multi} &\approx 40.22ps
 \end{aligned}$$

➤ Error Analysis For Multiplier

$$\begin{aligned}
 t_{simulation} &= 46.91ps \\
 t_{multi} &= 39.269ps \\
 error &= \left| \frac{39.269 - 46.91}{46.91} \right| \times 100 = 16.287\%
 \end{aligned}$$

6.4 Stage-by-Stage Delay Calculation for CLA Adder

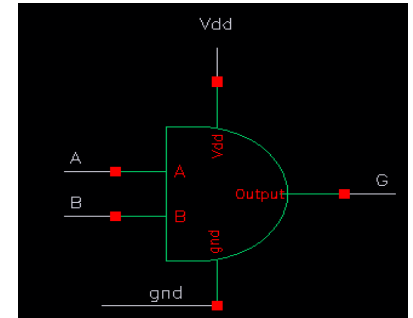
The critical path through the CLA Adder starts from the outputs of the two Barrel Shifters (B1<2> and B2<2>) and propagates through a chain of logic gates until reaching the final output SUM<4>. The stages that contribute actual delay are analyzed below, while stages that evaluate to a constant zero are identified and skipped as they contribute no switching delay.

6.4.1 [Step-by-Step Stage Delay Calculation]

➤ Stage 1 : AND Gate (G<2>)

Inputs: B1<2> and B2<2> → Output: G<2>

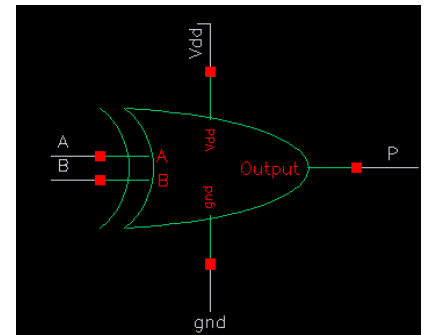
$$\begin{aligned} t_1 &= t_{int,AND} + 0.69 \times R_{AND} \times [C_{in}(OR) + C_{in}(AND)] \\ &= 16.57 \text{ ps} \end{aligned}$$



➤ Stage 2 : XOR Gate (P<2>)

Inputs: B1<2> and B2<2> → Output: P<2>

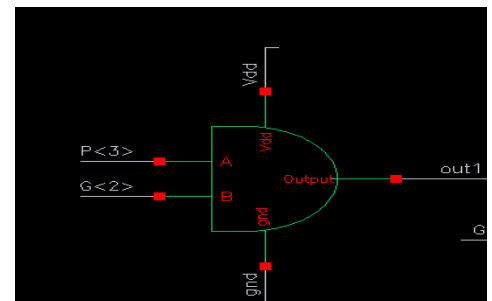
$$\begin{aligned} t_2 &= t_{int,XOR} + 0.69 \times R_{XOR} \times [C_{in}(XOR) + 3C_{in}(AND)] \\ &= 31.16 \text{ ps} \end{aligned}$$



➤ Stage 3 : AND Gate (out2)

Inputs: P<3> and G<2> → Output: out2

$$\begin{aligned} t_3 &= t_{int,AND} + 0.69 \times R_{AND} \times C_{in}(OR) \\ &= 14.57 \text{ ps} \end{aligned}$$

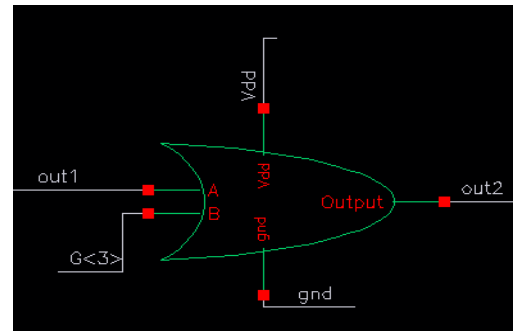


➤ Stage 4 : OR Gate (out2)

Inputs: out1 and G<3> → Output: out2

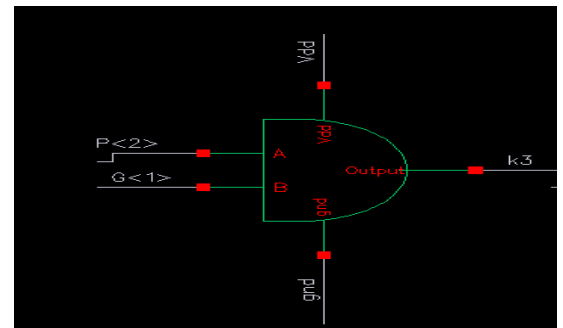
$$t_4 = t_{int,OR} + 0.69 \times R_{OR} \times C_{in}(OR)$$

$$= 14.57 \text{ ps}$$



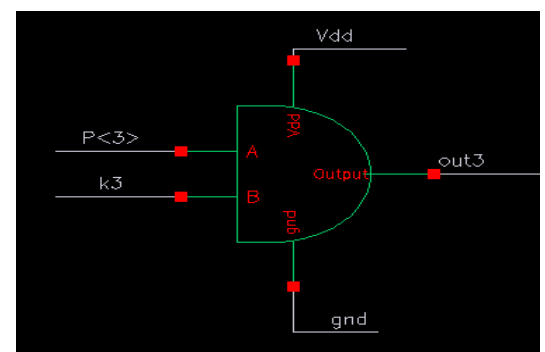
➤ Stage : G<1> = 0 → K3 always = 0 (No Delay)

Since G<1> = 0, K3 is always 0, contributing no delay.



➤ Stage : out3 = 0 (No Delay)

Output of AND gate (P<3>, K3) is always 0 → No Delay.

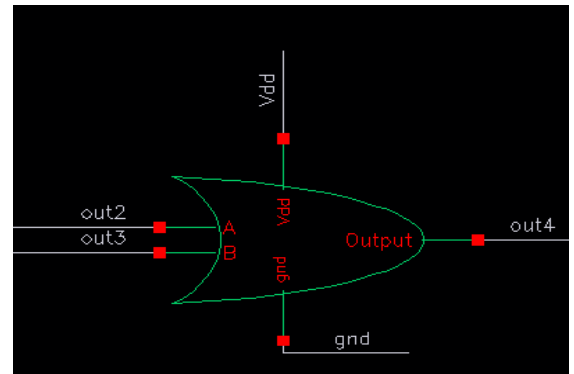


➤ Stage 5 : OR Gate (out4)

Inputs: out3 and out2 → Output: out4

$$t_5 = t_{int,OR} + 0.69 \times R_{OR} \times C_{in}(OR)$$

$$= 14.57 \text{ ps}$$



➤ Stage : K2 = 0 → K4 always = 0 (No Delay)

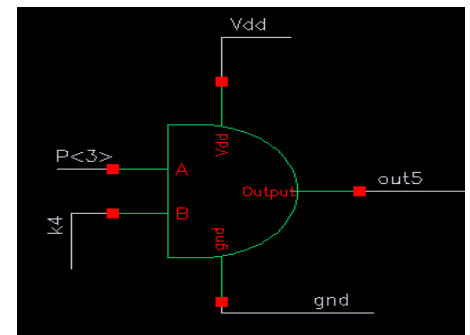
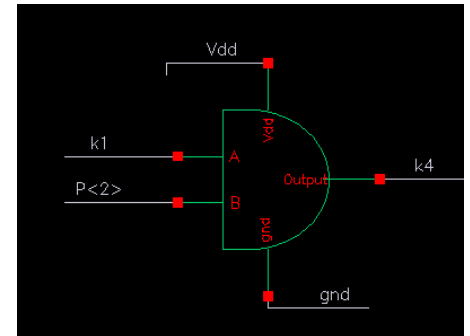
$$k2 = (k0) \cdot (P<1>) \cdot (P<0>)$$

$$k0 = (C0) \cdot (P<0>)$$

$$C0 = 0$$

$$\therefore k2 = 0, k5 \text{ always} = 0$$

out 7 → No Delay

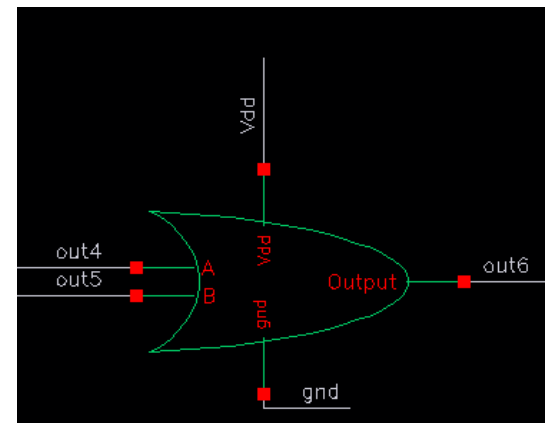


➤ Stage 6 : OR Gate (out6)

Inputs: out4 and out5 → Output: out6

$$t_6 = t_{int,OR} + 0.69 \times R_{OR} \times C_{in}(OR)$$

$$= 14.57 \text{ ps}$$



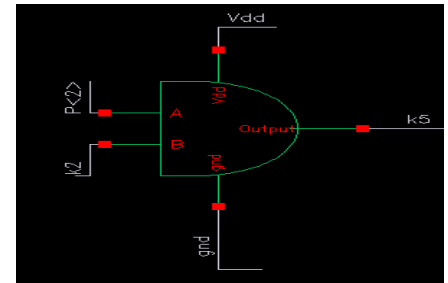
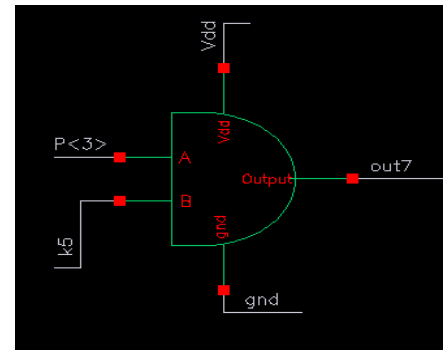
➤ Stage : k2 Evaluation

$$k2 = (k0) \cdot (P<1>) \cdot (P<0>)$$

$$k0 = (C0) \cdot (P<0>)$$

$$C0 = 0 \rightarrow k2 = 0 \rightarrow k5 \text{ always} = 0$$

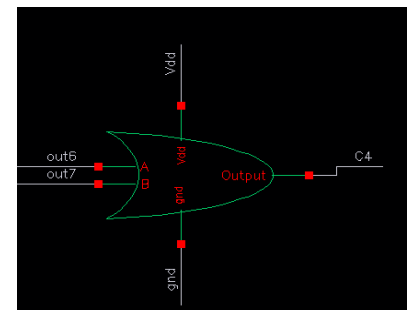
out 7 → No Delay



➤ Stage 7 : OR Gate (out 7, using C4)

Inputs: out7 and out6 → Output: C4

$$\begin{aligned} t_7 &= t_{int,OR} + 0.69 \times R_{OR} \times C_{in}(XOR) \\ &= 16.57 \text{ ps} \end{aligned}$$

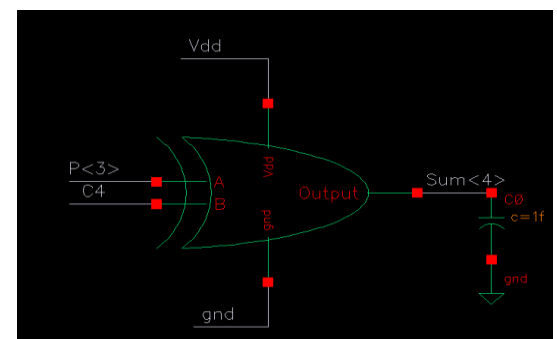


➤ Stage 8 : XOR Gate (SUM<4>)

Inputs: P<3> and C4

Output: SUM<4>,
with load capacitance $C_L = 1 \text{ fF}$

$$t_8 = t_{int,XOR} + 0.69 \times R_{XOR} \times C_l = 25 \text{ ps}$$



6.4.2 [Total CLA Adder Delay]

➤ Sum of 8-stage Delay

$$\begin{aligned}t_{adder} &= t_1 + t_3 + t_4 + t_5 + t_6 + t_7 + t_8 \\&= 16.57 + 14.57 + 14.57 + 14.57 + 14.57 + 16.57 + 25 \\&= 116.42 \text{ ps}\end{aligned}$$

➤ Error Analysis For CLA Adder

$$\begin{aligned}t_{simulation} &= 118.9 \text{ ps} \\t_{adder} &= 116.42 \text{ ps} \\error &= \left| \frac{116.42 - 118.9}{118.9} \right| \times 100 = 2.085 \%\end{aligned}$$

6.5 Total Perceptron Delay

➤ Sum of delays of (Multiplier + CLA)

$$\begin{aligned}t_{perceptron} &= t_{multiplier} + t_{adder} \\t_{perceptron} &= 116.42 + 39.269 = 155.689 \text{ ps}\end{aligned}$$

➤ Error Analysis For Full Perceptron

$$\begin{aligned}t_{simulation} &= 165.8 \text{ ps} \\t_{perceptron} &= 155.689 \text{ ps} \\error &= \left| \frac{155.689 - 165.8}{165.8} \right| \times 100 = 6.0983 \%\end{aligned}$$

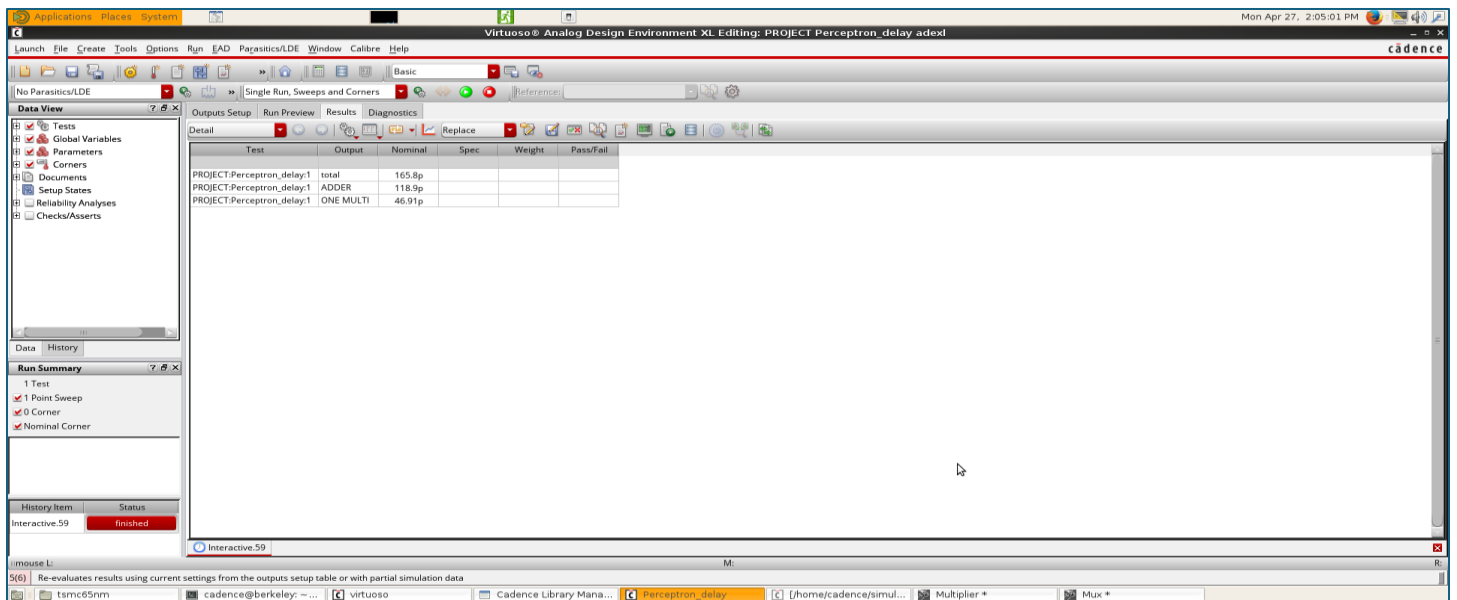
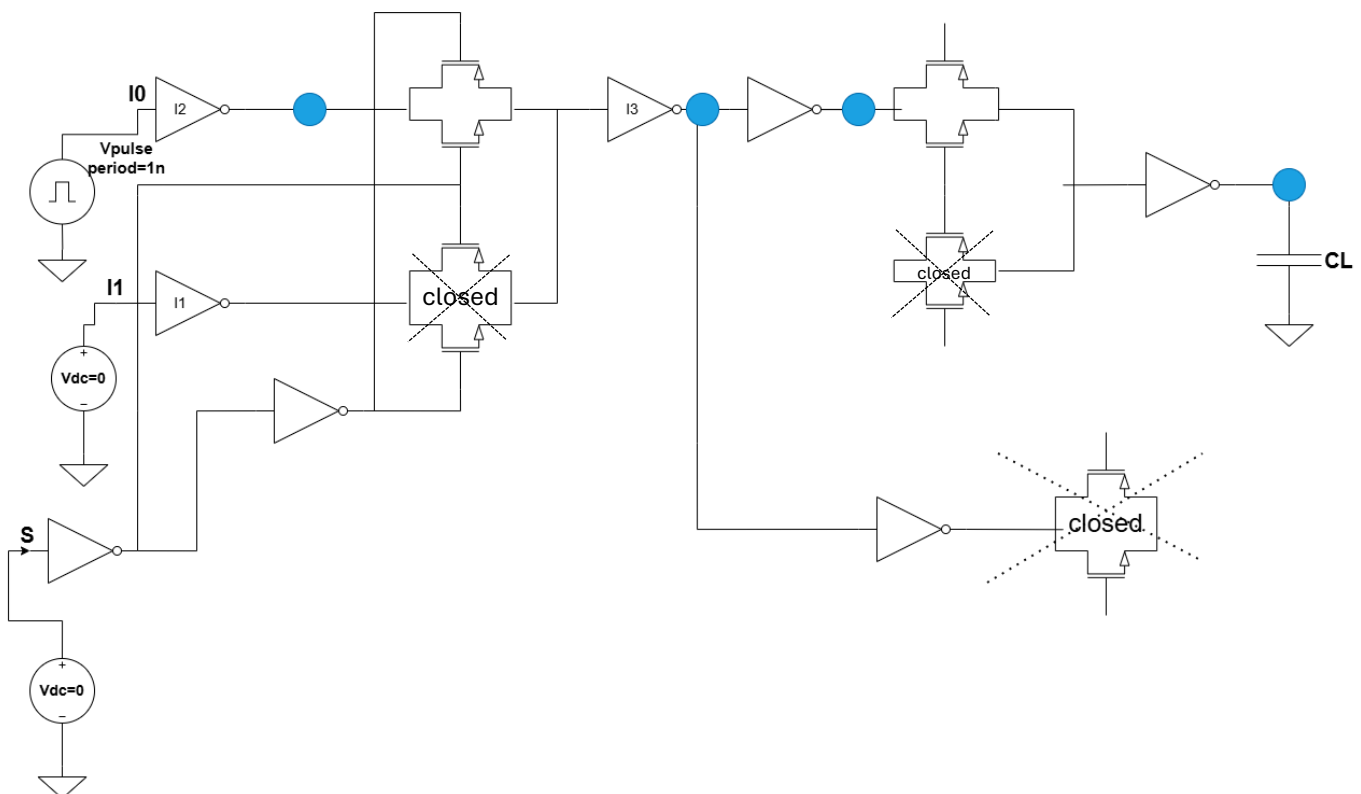


Figure 12: Cadence ADE Simulation Results (Perceptron Delay Measurements)

7. Hand-Analyzed and Simulated Power Consumption Results For The Multiplier



Blue circles represent the transition of bits which results in current from which the power was calculated.

$$\text{Power in Cadence} = I \times V_{DD} = 9.441666 \mu \times 1.2 = 11.33 \mu\text{Watt}$$

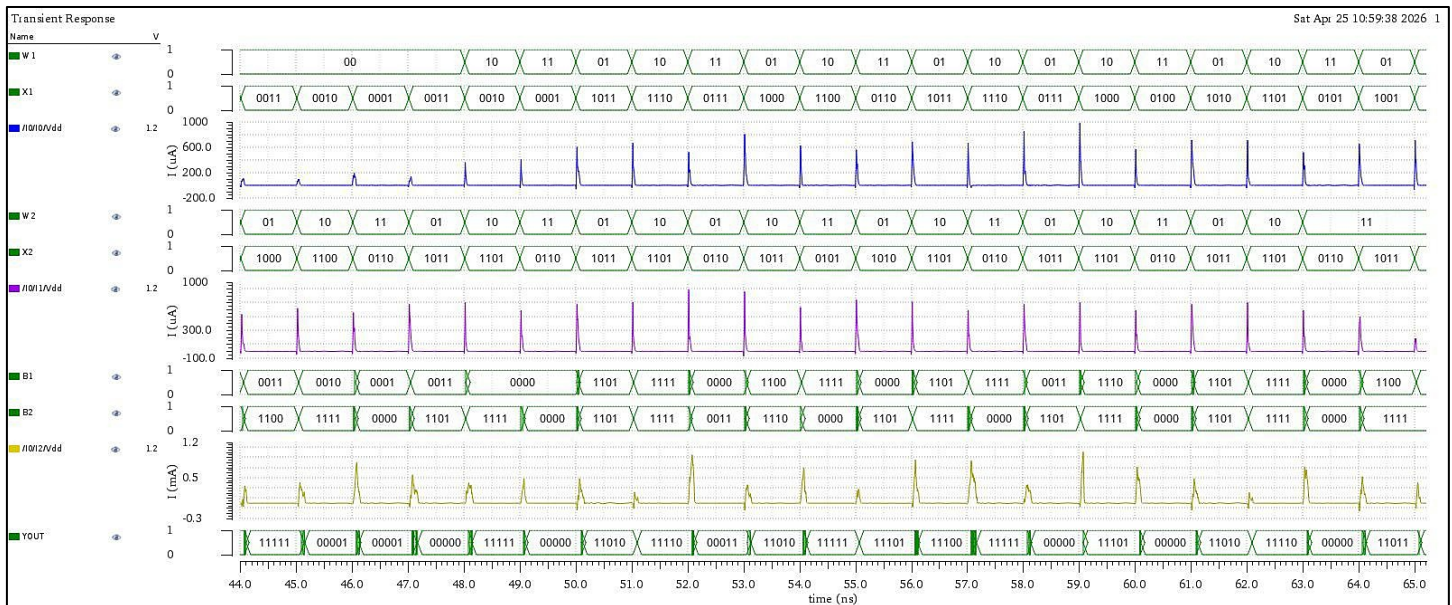
$$C_{total} = 2C_{dd} + C_{dd} + 2C_{gg} + C_{dd} + 2C_{g_{in,nand}} + C_{g_{in,nor}} + 2C_{dd} \\ = 7.1181 \text{ fF}$$

$$\text{Power}_{hand} = \alpha \times V_{DD}^2 \times C_{total} \times \text{freq} = 1.2^2 \times 7.1181 \times 10^{-15} \times 10^9 = 10.25 \mu\text{W}$$

$$\text{error} = \left| \frac{10.25 - 11.33}{11.33} \right| \times 100 = 9.532\%$$

$\alpha = 1$ is chosen to assume a logic transition occurs in every clock cycle

Test	Output	Nominal	Spec	Weight	Pass/Fail
PROJECT-Perceptron_delay:1 down multi	11.33u				
PROJECT-Perceptron_delay:1 adder	56.2u				
PROJECT-Perceptron_delay:1 total	67.76u				
PROJECT-Perceptron_delay:1 /0/1/0dd					
PROJECT-Perceptron_delay:1 /0/1/MINUS					
PROJECT-Perceptron_delay:1 /0/12/0dd					



In the waveforms generated through the simulation process, it can be noted that there is absolutely no static power consumption on each block within the circuit, and this is one of the key attributes of CMOS technology. As seen from the waveforms of current flow from VDD, it only spikes up when there are changes in the inputs, demonstrating that power is consumed exclusively through dynamic switching activity.